面试篇之场景题

01、开篇-面试那些事儿

现在的面试，在问到项目经验的时候，一般会让面试者先聊一下他们项目的整体架构【分布式还是单体】，讲一下用了哪些编程技术和核心的业务思路。比如面试者说项目用到了 Redis，面试官更关注的是你是在什么场景下用到了 Redis，以及应用到 Redis 的具体业务流程是什么，在当前业务中用到的是 Redis 何种数据结构，如何设计关键字 Key，以及原因。进而可能会问到如果 Redis 宕机，你的项目会受到什么影响？如何容灾等问题。也就是深挖细节。因为通过你回答这些问题，可以深入了解面试者对技术的掌握程度以及是否真正的将 Redis 用到了项目中。

那么如何讲解一个项目呢？我们可以通过以下几要点去讲解。

1、讲解项目的整体架构

2、讲解项目的业务流程和亮点业务，可以针对该业务描述下具体业务实现

3、讲解在项目实现过程中遇到的难点问题以及解决方案。

4、自己在项目中的贡献和积极态度。

面试篇之场景题

02、海量 QQ 去重

40 亿个 QQ 号如何去重？仅 1GB 内存

【腾讯】

解析：一个整数占 4 个字节，那么 40 亿个整数是：需要用 14.9GB 的内存，占用的空间是非常大的。那么如果用位图去解决，每一位代表一个数字【相当于一个桶】，一个整型 32 位，就可以代表 32 个数字，那么 40 亿个数字，就只需要用 40 亿+1 个位。也就是约等于：0.46GB 的内存就可以了。

```
public class MyBitmap {
    11 usages
    private long[] bits; // 使用 long 数组存储位数据
    1 usage
    public MyBitmap() {
        bits = new long[1]; // 初始数组长度为 1，可容纳 0-63
    }
```

```java
/**
 * 插入数据，将对应位设为1
 * @param num 要插入的非负整数
 */
3 usages
public void add(int num) {
    if (num < 0) {
        throw new IllegalArgumentException("Number must be non-negative");
    }
    int index = num / 64; // 计算数组下标
    if (index ⩾ bits.length) {
        expandCapacity( requiredSize: index + 1); // 需要扩容
    }
    int bitPosition = num % 64; // 计算位偏移
    bits[index] ⊨ (1L << bitPosition); // 设置对应位为1
}
```

```java
/**
 * 删除数据，将对应位设为0
 * @param num 要删除的非负整数
 */
1 usage
public void remove(int num) {
    if (num < 0) return;
    int index = num / 64;
    if (index ⩾ bits.length) return; // 超出当前存储范围

    int bitPosition = num % 64;
    bits[index] &= ~(1L << bitPosition); // 清除对应位
}
```

```java
/**
 * 检查数字是否存在
 * @param num 要检查的非负整数
 * @return 是否存在
 */
4 usages
public boolean contains(int num) {
    if (num < 0) return false;
    int index = num / 64;
    if (index >= bits.length) return false;

    int bitPosition = num % 64;
    return (bits[index] & (1L << bitPosition)) != 0;
}
```

```java
/**
 * 扩容数组以确保容量足够
 * @param requiredSize 需要的最小容量
 */
1 usage
private void expandCapacity(int requiredSize) {
    long[] newBits = new long[requiredSize];
    System.arraycopy(bits, srcPos: 0, newBits, destPos: 0, bits.length);
    bits = newBits;
}
```

```java
public static void main(String[] args) {
    MyBitmap bitmap = new MyBitmap();
    // 插入测试
    bitmap.add(5);
    bitmap.add(10);
    System.out.println("Contains 5? " + bitmap.contains(5));  // true
    System.out.println("Contains 10? " + bitmap.contains(10)); // true
    // 删除测试
    bitmap.remove( num: 5);
    System.out.println("After remove 5, contains 5? " + bitmap.contains(5)); // false
    // 扩容测试
    bitmap.add(128); // 需要扩容到长度 3
    System.out.println("Contains 128? " + bitmap.contains(128)); // true
    System.out.println("Current array length: " + bitmap.bits.length); // 3
}
```

面试篇之场景题

03、百万级别数据的 Excel 如何快速导入到数据库中

【阿里】

解析：从 Excel 中读取百万级数据并将其插入数据库时可能遇到的问题：

首先，数据正确性问题，读取和导入过程中会有各种潜在的问题，我们需妥善处理各类异常情况，在插入数据前，我们先进行格式检查，然后在插入过程中处理异常情况，如果单条数据插入失败，我们可以二次尝试，二次尝试仍然失败，需要记录日志，方便后续核对数据进行二次插入数据。导入数据过程中还需要注意数据可能出现重复的情况，我们可以采取设定唯一约束的方式来处理。当然也需要记录日志。

其次，就是内存溢出问题。

我们可以想一下，加载如此庞大的 Excel 数据到内存可能会导致内存溢出，所以我们需要注意内存管理。所以我们不能一次性将数据读取到内存，需要分批处理数据。我们可以使用 Apache POI 的事件模型（SAX）来处理 Excel，这种方式适合处理大文件，因为它不会将整个文件加载到内存中。，它会按行读取数据并解析，这样就大大的解放了内存的压力。

还有就是性能瓶颈问题

针对百万级数据的处理，单线程显然效率很低，所以我们采取多线程处理数据。我们可以将多线程操作应用在两种场景下，一种是多线程读取文件操作。另一个是线程插入数据。当然在数据插入的时候，除了使用多线程技术外，还可以使用批量插入数据的功能更进一步提升速度。可以通过 values (),(),() 的方式一次插入多行数据。

```
CREATE TABLE bookinfo (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    author VARCHAR(100) NOT NULL,
    bookdesc TEXT,
    price DECIMAL(10,2) NOT NULL
);
```

04、从 mysql 千万条数据中搜索到指定数据

【字节】

解析：首先我们先创建一张千万级数据的表 test_data。

代码如下：

```
CREATE TABLE test_data ( id INT AUTO_INCREMENT
PRIMARY KEY, random_str VARCHAR(20), create_date
DATETIME, num_value INT );
```

然后我们插入数据：

```
-- 创建存储过程（示例）
DELIMITER $$
CREATE PROCEDURE generate_data(IN total INT)
BEGIN
    DECLARE i INT DEFAULT 0;
    START TRANSACTION;
    WHILE i < total DO
```

```
            INSERT INTO test_data (random_str,
create_date, num_value)
            VALUES (
                CONCAT('DATA_',
LPAD(FLOOR(RAND()*1000000), 6, '0')),
                NOW() - INTERVAL FLOOR(RAND()*3650)
DAY,
                FLOOR(RAND()*10000)
            );
            SET i = i + 1;
            IF i % 1000 = 0 THEN
                COMMIT;
                START TRANSACTION;
            END IF;
        END WHILE;
        COMMIT;
END$$
DELIMITER ;

-- 执行存储过程
CALL generate_data(10000000);
```

接下来我们来执行如下 SQL：

```
select * from test_data limit 10000,100;
```

我们可以从表中检索到 100 条记录。

优化方案：

（1）使用列名替换*，让数据库引擎省去转换列名的代价，提升查询效率。

```
select id,random_str,create_date,num_value from
test_data limit 10000,100;
```

（2）使用限定 id 的方式进行查询，通过使用索引进行范围查找，提升查询效率。

```
select id,random_str,create_date,num_value from
test_data where id>10000 limit 100;
```

（3）通过查询业务需求列，减少数据传输量，提升查询效率。

```
select id,random_str from test_data where id>10000
limit 100;
```

（4）注意索引的有效性，应尽量避免在 where 子句中对字段进行函数操作，这将导致引擎放弃使用索引而进行全表扫描。

```
SELECT * FROM book.test_data
where    YEAR(create_date) = 2023;【不推荐】
```

SELECT * FROM book.test_data where create_date BETWEEN '2023-01-01' AND '2023-12-31';【优化后】

面试篇之场景题

05、为什么复杂的架构一定要做分层设计?

【腾讯】

解析：分层思想在软件工程中是一种常见的设计方式，它是将整体系统拆分成 N 个层次，每个层次有独立的职责，多个层次协同提供完整的功能。软件架构整体上会被分为三层：UI 层、业务逻辑层、数据访问层。

UI 层：就是进行数据展示以及和用户进行交互的。

业务逻辑层：负责业务逻辑的实现，在业务逻辑层，可能会综合调度多个数据访问层，做综合数据处理。

数据访问层：主要和数据库进行通信。

分层架构的优点：

（1）职能分离，隔离关注点，每个人关注自己所在分层的任务就 ok 了。

（2）解耦合，方便系统扩展。

当项目分层后，每层都仅能和自己相关的层做数据交互，不能跨层调度，这样就做到了解耦，同时也方便业务的扩展。我们可以将数据访问和业务逻辑层独立部署。

面试篇之场景题

06、什么是网络四元组

【京东】

网络四元组是 TCP 协议中用于唯一标识一个连接的核心概念，它由源 IP 地址、目标 IP 地址、源端口号、目标端口号四个要素组成。

（1）源 IP 地址：发起连接的客户端 IP 地址。

（2）目标 IP 地址：服务端的 IP 地址。

（3）源端口号：客户端随机分配的端口号（范围：1024-65535）。

（4）目标端口号：服务端监听的固定端口号（如 HTTP 的 80、HTTPS 的 443）

那么网络四元组的核心作用主要是什么呢?其实网络四元组的核心作用有主要有两个，一个是唯一标识连接，一个是全双工通信。我们先来聊一下唯一标识连接。通过四元组区分同一服务端上的不同客户端连接。例如，同一客户端主机通过不同端口访问同一服务端时，源端口不同，形成不同的四元组。然后就是支持全双工通信：TCP 允许双向数据传输，四元组确保数据在客户端与服务端之间准确路由。

最后我们聊一下四元组的工作原理

```
客户端A (IP: 10.23.15.3)                    服务端 (IP: 192.168.8.135, 端口: 8080)
├─ 连接1：源端口59461 → 四元组 (10.23.15.3:59461, 192.168.8.135:8080)
├─ 连接2：源端口59462 → 四元组 (10.23.15.3:59462, 192.168.8.135:8080)
└─ 连接3：源端口59463 → 四元组 (10.23.15.3:59463, 192.168.8.135:8080)
```

面试篇之场景题

07、SpringBoot 自动装配的原理

【小米】

（1）首先，我们先看下什么是自动装配，自动装配是 Spring Boot 的核心特性，其核心目标是"约定优于配置"，通过智能判断项目依赖和环境，自动将组件注册到 IOC 容器，开发者无需手动编写 XML 或配置类。

传统配置：

开发者 → 手动编写 Bean 定义 → 注册到 IOC 容器

自动装配：

引入 Starter → Spring Boot → @EnableAutoConfiguration → 加载 spring.factories → 条件筛选→动态注册 Bean

通过以上机制，Spring Boot 实现了“零配置”的开发体验，显著提升开发效率。

（2）自动装配的核心机制

三大核心组件：@EnableAutoConfiguration、spring.factories 文件、条件注解（Conditional）

@EnableAutoConfiguration：

启动类上的@SpringBootApplication 包含此注解，触发自动装配流程。

spring.factories 文件：

通过 SpringFactoriesLoader 加载所有 spring.factories 中定义的配置类

在 META-INF 目录下定义自动配置类全限定名。

条件注解（Conditional）

根据当前环境（类路径、已有 Bean、配置属性）筛选有效配置类。

例如，仅当引入 JDBC 驱动时，DataSourceAutoConfiguration 才会生效。

经过上面三个步骤，我们就可以注册 Bean 了。

注册 Bean：

符合条件的配置类中定义的 Bean 被实例化并注册到 IOC 容器。

（3）Redis 自动装配

引入依赖

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

Spring Boot 检测到 RedisTemplate 类存在，自动配置连接工厂和 RedisTemplate Bean。

直接注入使用

```
@Configuration
public class T1 {
    @Autowired
    private RedisTemplate<String, String> redisTemplate;

}
```

面试篇之场景题

08、在 2G 大小的文件中，找出高频 top100 的单词

【腾讯】

1. 分块读取（内存控制）

我们可以按 4MB 为单位分块读取文件，使用 BufferedReader 逐行读取避免内存溢出。我们也可以动态调整分块大小（根据实际内存情况）。

2. 并行处理（性能优化）

使用线程池处理每个分块，每个线程独立统计局部词频，合并结果时使用 ConcurrentHashMap 保证线程安全。

3.全局合并（高效聚合）

使用 Map.merge 方法进行原子累加，同步代码块保证并发安全

4. Top K 筛选（堆排序优化）

维护大小为 100 的最小堆，最终排序输出结果。

具体过程就是如果遍历到的词的出现次数大于堆顶上词的出现次数，那么可以用新遍历到的词替换堆顶的词，然后重新调整这个堆为小顶堆。当遍历完所有的文件后，这个小顶堆中的词就是出现频率最高的 100 个词。

1. 分块读取 → 2. 局部统计 → 3. 全局合并 → 4. 堆排序筛选

具体代码如下：

```java
public class TopWordCounter {
    // 配置参数
    1 usage
    private static final int CHUNK_SIZE = 4 * 1024 * 1024; // 4MB/块
    1 usage
    private static final int TOP_K = 100;
    1 usage
    private static final Pattern WORD_PATTERN = Pattern.compile( regex: "[^a-zA-Z']+");

    // 全局统计结果
    3 usages
    private final Map<String, Long> globalCount = new ConcurrentHashMap<>();
```

```java
public static void main(String[] args) throws Exception {
    long start = System.currentTimeMillis();
    new TopWordCounter().processFile( filePath: "D:\\Java\\2025\\big\\big.txt");
    System.out.printf("Total time: %dms", System.currentTimeMillis() - start);
}
```

```java
// 主处理流程
1 usage
public void processFile(String filePath) throws Exception {
    ExecutorService executor = Executors.newFixedThreadPool
            (Runtime.getRuntime().availableProcessors());

    try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
        List<String> chunk = new ArrayList<>();
        String line;
        int currentSize = 0;

        // 分块读取
        while ((line = br.readLine()) != null) {
            chunk.add(line);
            currentSize += line.length();
            if (currentSize >= CHUNK_SIZE) {
                executor.submit(new WordCountTask(new ArrayList<>(chunk)));
                chunk.clear();
                currentSize = 0;
            }
```

```
            if (!chunk.isEmpty()) {
                executor.submit(new WordCountTask(chunk));
            }
        }

        // 等待所有任务完成
        executor.shutdown();
        executor.awaitTermination( timeout: 1, TimeUnit.HOURS);

        // 提取Top K结果
        PriorityQueue<Map.Entry<String, Long>> heap = new PriorityQueue<>(
            Comparator.comparingLong(Map.Entry::getValue)
        );
```

```
        for (Map.Entry<String, Long> entry : globalCount.entrySet()) {
            if (heap.size() < TOP_K) {
                heap.offer(entry);
            } else if (entry.getValue() > heap.peek().getValue()) {
                heap.poll();
                heap.offer(entry);
            }
        }

        // 输出结果
        List<Map.Entry<String, Long>> result = new ArrayList<>(heap);
        result.sort((a, b) -> Long.compare(b.getValue(), a.getValue()));

        System.out.println("Top 100 Words:");
        result.forEach(e -> System.out.printf("%s : %d%n", e.getKey(), e.getValue()));
    }
```

```
private class WordCountTask implements Runnable {
    2 usages
    private final List<String> chunk;
    2 usages
    public WordCountTask(List<String> chunk) { this.chunk = chunk; }
    @Override
    public void run() {
        Map<String, Long> localCount = new HashMap<>();

        for (String line : chunk) {
            String[] words = WORD_PATTERN.split(line.toLowerCase());
            for (String word : words) {
                if (!word.isEmpty()) {
                    localCount.put(word, localCount.getOrDefault(word, defaultValue: 0L) + 1);
                }
            }
        }
```

```
            // 合并到全局统计
            synchronized (globalCount) {
                localCount.forEach((k, v) ->
                    globalCount.merge(k, v, Long::sum)
                );
            }
        }
    }
```

面试篇之场景题

09、Redis 缓存击穿问题及解决方案

【阿里】

Redis 缓存击穿（Cache Breakdown）是分布式系统中常见的高并发问题，其核心特点是某个热点 Key 在缓存失效的瞬间，大量并发请求直接穿透缓存层，直接冲击数据库，就像在一个屏障上凿开了一个洞。

缓存击穿一般有以下几个原因：

（1）热点 Key 特性：例如明星绯闻、秒杀商品等高频访问数据。

（2）缓存过期机制：Key 设置了固定过期时间，到期后缓存失效。

（3）高并发请求：缓存失效瞬间，大量请求同时到达数据库。

解决缓存击穿一般有三种方法：

（1）互斥锁（Mutex Lock）

通过分布式锁（如 Redis 的 SETNX）让第一个请求重建缓存，其他请求等待。

```
public String getData(String key) {
    String value = redis.get(key);
    if (value == null) {
        if (redis.setnx(key + ":mutex", "1", 60)) { // 获取锁
            value = db.query(key); // 查数据库
            redis.set(key, value, 3600);
            redis.del(key + ":mutex"); // 释放锁
        } else {
            Thread.sleep(100); // 等待重试
            return getData(key); // 递归调用
        }
    }
    return value;
}
```

（2）逻辑过期（Logical Expiration）

缓存永不过期，但 Value 中存储逻辑过期时间，异步更新。

```
class CacheData {
    String data;
    long expireTime; // 逻辑过期时间
}

public String getData(String key) {
    CacheData cacheData = redis.get(key);
    if (cacheData == null) {
        return db.query(key); // 首次加载
    }
    if (System.currentTimeMillis() > cacheData.expireTime) {
        // 异步更新
        executor.submit(() -> {
            String newData = db.query(key);
            redis.set(key, new CacheData(newData, System.currentTimeMillis() + 3600000));
        });
    }
    return cacheData.data;
}
```

（3）缓存预热（Cache Warm-Up）

在系统启动或低峰期提前加载热点数据。监控历史访问数据，预测热点 Key 并提前缓存。

面试篇之场景题

10、缓存雪崩问题及解决方案

【字节】

缓存雪崩（Cache Avalanche）是分布式系统中另一个典型的高并发问题，其核心特征是大量缓存 Key 在同一时间集中失效或 Redis 服务整体宕机，导致海量请求直接穿透到数据库，引发数据库瞬时压力激增甚至崩溃。

缓存雪崩的常见原因

（1）批量 Key 同时过期

例如：定时任务批量刷新缓存，设置相同的 TTL（如每天 0 点统一过期）。

（2）Redis 服务不可用

如主从切换失败、集群节点宕机、网络故障等。

（3）冷启动未预热

新系统上线或重启后，缓存中无数据，首次请求直接压垮数据库。

解决方案：

1. 分散 Key 过期时间

核心思想：避免大量 Key 同时失效，通过随机值打散过期时间。

```
// 基础过期时间 + 随机偏移量（如0~300秒）
int baseTTL = 3600;
int randomOffset = new Random().nextInt(300);
redis.set(key, value, baseTTL + randomOffset);
```

2. 多级缓存架构

核心思想：通过本地缓存（如 Caffeine）+ Redis + 数据库构建多级缓存，降低单点风险。

```
public String getData(String key) {
    // 1. 查询本地缓存
    String value = localCache.get(key);
    if (value != null) return value;

    // 2. 查询Redis
    value = redis.get(key);
    if (value != null) {
        localCache.put(key, value); // 回填本地缓存
        return value;
    }

    // 3. 查数据库并重建缓存
    value = db.query(key);
    redis.set(key, value, 3600 + randomOffset);
    localCache.put(key, value);
    return value;
}
```

3.熔断降级与限流

熔断降级：当数据库压力超过阈值时，直接拒绝部分请求，返回默认值或错误页。

可以使用 Hystrix、Sentinel 在网关或服务层限制并发请求数。

其实集中过期，倒不是非常致命，比较致命的缓存雪崩，是缓存服务器某个节点宕机或断网。

因为自然形成的缓存雪崩，一定是在某个时间段集中创建缓存，那么那个时候数据库能顶住压力，这个时候，数据库也是可以顶住压力的。无非就是对数据库产生周期性的压力而已。而缓存服务节点的宕机，对数据库服务器造成的压力是不可预知的，很有可能瞬间就把数据库压垮。

面试篇之场景题

11、缓存穿透及解决方案

【字节】

缓存穿透（Cache Penetration）是指频繁查询数据库中不存在的数据，导致请求绕过缓存直接冲击数据库。与缓存击穿（热点 Key 失效）、缓存雪崩（大量 Key 失效）不同，缓存穿透的核心问题是恶意或无效请求对系统的持续攻击。

缓存穿透的原因：

（1）恶意攻击：攻击者故意构造大量不存在的数据 ID（如负值、超大数值）。

（2）业务设计缺陷：未对请求参数做合法性校验（如非法的用户 ID、商品 ID）。

我们知道正常的代码流程如下：

（1）参数传入对象主键 ID

（2）根据 key 从缓存中获取对象

（3）如果对象不为空，直接返回

（4）如果对象为空，进行数据库查询

（5）如果从数据库查询出的对象不为空，则放入缓存（设定过期时间）

想象一下这个情况，如果传入的参数为-1，会是怎么样？这个-1，就是一定不存在的对象。就会每次都去查询数据库，而每次查询都是空，每次又都不会进行缓存。假如有恶意攻击，就可以利用这个漏洞，对数据库造成压力，甚至压垮数据库。即便是采用 UUID，也是很容易找到一个不存在的 KEY，进行攻击。

解决方案：

1. 布隆过滤器（Bloom Filter）

核心思想：利用概率型数据结构快速判断 Key 是否存在，拦截无效请求。

```
// 初始化布隆过滤器（需提前加载所有合法Key）
BloomFilter<String> bloomFilter = BloomFilter.create(Funnels.stringFunnel(), 1000000, 0.01);

public String getData(String key) {
    if (!bloomFilter.mightContain(key)) { // 判断Key是否存在
        return "无效请求";
    }
    String value = redis.get(key);
    if (value == null) {
        value = db.query(key);
        redis.set(key, value, 3600);
    }
    return value;
}
```

2. 空值缓存（Cache Null）

核心思想：即使数据库无数据，也缓存空值（如 NULL），避免重复查询数据库。

```
public String getData(String key) {
    String value = redis.get(key);
    if (value != null) {
        return "NULL".equals(value) ? null : value; // 处理空值
    }
    value = db.query(key);
    if (value == null) {
        redis.set(key, "NULL", 60); // 空值缓存60秒
        return null;
    }
    redis.set(key, value, 3600);
    return value;
}
```

3. 请求参数校验

核心思想：在业务层拦截非法请求参数。

格式校验：检查 ID 是否为数字、长度是否符合规则。

范围校验：例如用户 ID 必须大于 0，商品 ID 必须为 6 位字母数字组合。

业务规则校验：如查询时间范围不超过 3 个月。

```
public String getData(String key) {
    if (!isValidKey(key)) { // 校验Key合法性
        throw new IllegalArgumentException("非法参数");
    }
    // 正常查询逻辑
}
```

面试篇之场景题

12、多线程 Future 的用法

【阿里】

在并发编程的时候，一般使用 runnable，然后扔给线程池完事，这种情况下不需要线程的结果。所以 run 的返回值是 void 类型。如果是一个多线程协作程序，比如斐波那契数据列，1,1,2,3,5,8...使用多线程来计算。但后者需要前者的结果，就需要用 callable 接口了。callable 用法和 runnable 一样，只不过调用的是 call 方法，该方法有一个泛型返回值类型，你可以任意指定。线程是属于异步计算模型，所以你不可能直接从别的线程中得到函数的返回值。这个时候，Future 就出场了。Future 可以监视目标线程调用 call 的情况，当你调用 Future 的 get()方法以获得结果时，当前线程就开始阻塞，直接 call 方法结束返回结果。

runnable 接口实现的没有返回值的并发编程

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class FutureTest01 {
    1 usage
    public static class Task implements Runnable{
        @Override
        public void run() { System.out.println("run"); }
    }

    public static void main(String[] args) {
        ExecutorService es= Executors.newCachedThreadPool();
        for (int i=0;i<100;i++){
            es.submit(new Task());
        }
    }
}
```

callable 实现的存在返回值的并发编程(call 的返回值 String 受泛型的影响)

```java
public class FutureTest02 {
    1 usage
    public static class Task implements Callable<String> {
        @Override
        public String call() throws Exception {
            System.out.println("run");
            return "something";
        }
    }
    public static void main(String[] args) {
        ExecutorService es= Executors.newCachedThreadPool();
        for (int i=0;i<100;i++){
            es.submit(new Task());
        }
    }
}
```

同样是 callable，使用 Future 获取返回值

```java
public class FutureTest03 {
    1 usage
    public static class Task implements Callable<String> {
        @Override
        public String call() throws Exception {
            System.out.println("run");
            return "something";
        }
    }
    public static void main(String[] args) throws ExecutionException, InterruptedException,
            TimeoutException {
        ArrayList<Future<String>> results=new ArrayList<Future<String>>();
        ExecutorService es= Executors.newCachedThreadPool();
        for (int i=0;i<100;i++){
            results.add(es.submit(new Task()));
        }
        for (Future<String> result:results){
            System.out.println(result.get());
        }
    }
}
```

```java
public class FutureBasicDemo {
    public static void main(String[] args) {
        // 创建线程池（核心线程数=2）
        ExecutorService executor = Executors.newFixedThreadPool( nThreads: 2);
        // 提交 Callable 任务
        Future<Integer> future = executor.submit(() -> {
            TimeUnit.SECONDS.sleep( timeout: 2); // 模拟耗时操作
            return 42; // 返回计算结果
        });
        try {
            Integer result = future.get( timeout: 3, TimeUnit.SECONDS);// 阻塞获取结果（最长等待3秒）
            System.out.println("计算结果: " + result);
        } catch (TimeoutException e) {
            System.err.println("任务超时未完成");
        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        } finally {
            executor.shutdown(); // 关闭线程池
        }
    }
```

面试篇之场景题

13. SpringMvc 的优点和缺点

【小米】

我们先讲解以下 SpringMvc 的优点:

1. 松耦合与模块化设计

MVC 分层架构：将模型（Model）、视图（View）、控制器（Controller）职责分离，降低代码耦合度，提升可维护性和可测试性

依赖注入（DI）：通过 Spring 容器管理对象生命周期和依赖关系，减少硬编码依赖，增强模块化设计

2.可扩展性强

支持多种视图技术：如 JSP、Thymeleaf、FreeMarker 等，满足不同项目的视图需求

插件化扩展：通过拦截器（Interceptor）、自定义数据绑定器等扩展点，可快速集成第三方库或自定义功能

3.配置方式多样

支持注解驱动（如 @Controller、@RequestMapping）和 XML 配置，适应不同复杂度项目的需求

4.强大的数据请求能力

数据绑定与验证：自动绑定请求参数到对象，支持 JSR-303 标准的数据校验（如 @Valid 注解），减少手动处理代码

5.与 Spring 生态无缝集成

一站式解决方案：与 Spring Boot、Spring Security、Spring Data 等模块深度集成，支持从数据层到表现层的全栈开发

事务管理与 AOP：通过 Spring 的声明式事务和面向切面编程，简化复杂业务逻辑的实现

6.高级功能支持

全局异常处理：通过 @ControllerAdvice 和 @ExceptionHandler 统一处理异常，提升代码健壮性。

7.国际化与本地化：支持多语言资源文件加载和动态语言切换，适应全球化需求 58。

高性能优化：通过缓存机制和异步请求处理（如 @Async）提升性能

当然 SpringMvc 也存在缺点：

1. 学习曲线陡峭

复杂概念较多：如依赖注入、AOP、拦截器等概念对新手不友好，需深入理解 Spring 生态

配置复杂度高：早期版本依赖 XML 配置，虽 Spring Boot 简化了配置，但复杂场景仍需手动调优

2. 性能开销

反射与代理机制：依赖注入和动态代理带来一定的性能损耗，尤其在超高并发场景下可能成

为瓶颈

视图层效率问题：频繁的模型数据访问和视图渲染可能影响响应速度

3. 不适合小型项目

过度设计风险：分层架构和模块化设计在小型项目中可能导致代码冗余，开发效率降低

依赖较重：引入 Spring 生态的多个模块会增加项目依赖和启动时间

Spring MVC 适合中大型企业级项目，尤其是需要高扩展性、复杂业务逻辑和深度集成 Spring 生态的场景。其优势在于模块化设计和灵活性，但学习成本和性能开销可能成为小项目的负担。对于追求快速开发的场景，Spring Boot + Spring MVC 的组合能显著简化配置，平衡功能与效率。

面试篇之场景题

14. 有没有碰到过执行计划不一致的情况，同一条 SQL 不同的执行场景用的索引不同，如何强制使用某个索引

【百度】

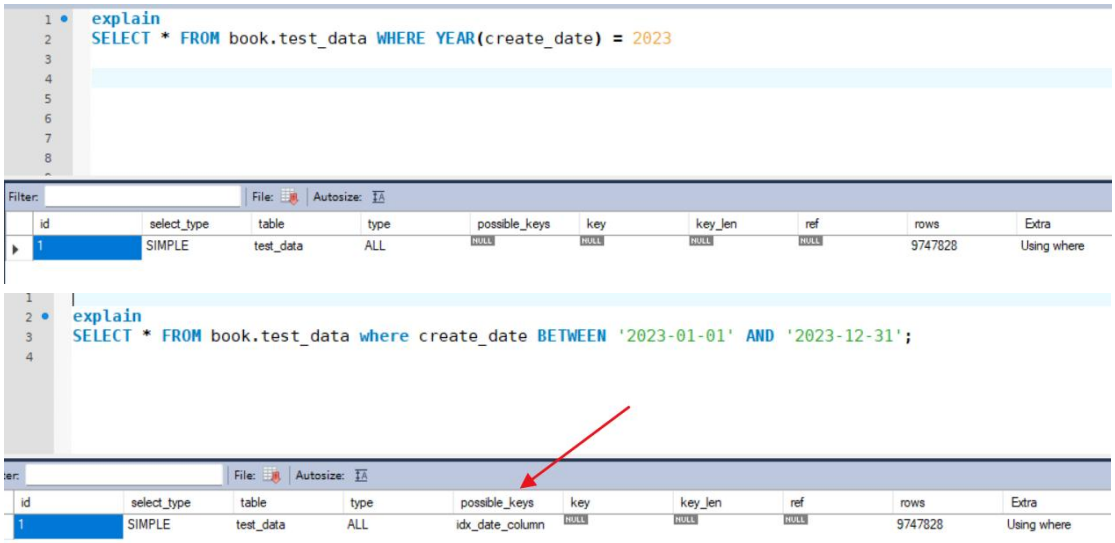
在实际工作中，我遇到过因统计信息不准确或数据分布变化导致同一 SQL 选择不同索引的情况。为解决此类问题，首先应通过 EXPLAIN 分析执行计划差异的根本原因。若需强制使用索引，可采用数据库特定的语法（如 MySQL 的 FORCE INDEX 或 Oracle 的 Hint）。但强制索引应作为最后手段，优先考虑优化查询逻辑、更新统计信息或设计覆盖索引。同时，需在测试环境充分验证并持续监控生产环境性能，避免因数据变化引发新的性能问题。

1、优先优化查询与索引

重写 SQL 使其更符合索引扫描条件（如避免对索引列使用函数 WHERE YEAR(create_date) = 2023 → WHERE create_date BETWEEN '2023-01-01' AND '2023-12-31'）。

2、逐步验证

使用 EXPLAIN 或 EXPLAIN ANALYZE 对比不同索引的执行计划



3、强制索引

test_data 表在测试环境使用时间索引，生产环境却走主键索引：

```
-- 原 SQL
SELECT * FROM orders WHERE create_date BETWEEN '2023-01-01' AND '2023-12-31';
-- 强制索引方案
SELECT * FROM orders FORCE INDEX(idx_create_date)
WHERE create_date BETWEEN '2023-01-01' AND '2023-12-31';
```

面试篇之场景题

15、如果查询优化器选错了索引怎么办

【腾讯】

在数据库优化中，查询优化器选择错误索引是常见问题，尤其是在数据分布变化或统计信息不准确时。首先呢我们先进行问题诊断：

（1）确认执行计划异常

```
explain
SELECT * FROM book.test_data where create_date>'2023-01-01'
```

确认实际使用的索引

（2）检查统计信息

```
-- MySQL 查看索引统计信息
SHOW INDEX FROM test_data;
```

解决方案：

1. 主动干预执行计划

强制指定索引，明确知道某个索引更优且需快速修复线上问题

```
SELECT * FROM test_data FORCE INDEX (idx_date_column) WHERE create_date > '2023-01-01';
```

2. 优化索引设计

创建覆盖索引，包含查询中所有需要字段（避免回表），使优化器更倾向选择该索引。

```
CREATE INDEX idx_covering ON orders (status, create_time, amount);
```

合并冗余索引，若存在单列索引 idx_status 和 idx_create_time，可合并为联合索引：

```
DROP INDEX idx_status, idx_create_time;
CREATE INDEX idx_composite ON orders (status, create_time);
```

3.查询重写优化

消除索引使用障碍

```
-- 原始SQL（无法使用索引）
SELECT * FROM book.test_data where  YEAR(create_date) = 2023;

-- 优化后（范围查询可利用索引）
SELECT * FROM book.test_data where create_date BETWEEN '2023-01-01' AND '2023-12-31';
```

拆分复杂查询

```sql
-- 原始SQL（多条件混杂）
SELECT * FROM orders
WHERE (status = 'pending' OR total_amount > 1000) AND create_time > '2023-01-01';

-- 拆分为UNION查询
SELECT * FROM orders WHERE status = 'pending' AND create_time > '2023-01-01'
UNION ALL
SELECT * FROM orders WHERE total_amount > 1000 AND create_time > '2023-01-01';
```

面试篇之场景题

16、当给第三方提供接口调用，需要注意哪些事情?

【快手】

（1）首先我们需要提供的接口需要遵循 RESTful 规范，使用资源化 URL（如 /v1/users/{id}）和 HTTP 方法（GET/POST/PUT/DELETE）。数据格式统一使用 json 格式。这几乎是行业内规范。还有的接口需要做到优雅降级，在高负载时关闭非核心功能，确保核心接口可用。再有就是需要提供接口文档以及 SDK 简化集成。

（2）在基础的接口设计好后，我们需要考虑安全性，使用 API Key、OAuth 2.0、JWT 等确保调用方身份合法，另外还需要考虑敏感数据脱敏：接口返回的敏感信息（如手机号、身份证）需加密或脱敏。第三就是考虑输入校验与防护。这里的校验是指参数

校验。参数校验：校验参数类型、长度、格式（如正则表达式），避免 SQL 注入、XSS 攻击。最后就是注意限流。限流策略：按 IP、用户或接口维度限制 QPS

（3）性能：性能的保证主要来源于缓存与异步

缓存优化：对高频读操作使用 Redis 或 Memcached 缓存，设置合理 TTL。

异步处理：耗时操作（如文件处理）通过消息队列（Kafka/RabbitMQ）异步化。

（4）日志的记录：

结构化日志：输出 JSON 格式日志，集成 ELK（Elasticsearch+Logstash+Kibana）分析。敏感日志过滤：避免记录密码、密钥等敏感信息。

终极目标是为了实现安全可靠、易于集成、高效稳定的 API 服务。

面试篇之场景题

17、设计 redis 的 key 和 value ，有哪些原则?

【快手】

首先，我们先来探讨下 key 的设计原则，key 的设计原则需要保证以下三点：

（1）可读性与规范性

使用统一的命名格式（如 业务模块:实体类型:唯一标识），例如：

user:profile:1001　　# 用户 ID 为 1001 的个人信息

order:status:20230801　# 订单日期为 20230801 的状态

需要注意避免特殊字符（如空格、引号），推荐使用 _ 或 : 分隔层级。

（2）避免过长 Key

Key 越长，内存占用越高，查询效率可能下降。

差：冗余信息过多

user_account_details_for_user_id_1001_in_2023

优：简洁明确

user:acc:1001:2023

（3）控制 Key 数量

Key 过多会导致内存碎片和管理复杂度上升，合理使用复合数据结构（如 Hash、ZSet）减少 Key 数量。

接下来我们看下 Value 的设计原则：

（1）选择合适的数据结构

根据业务需求选择最优结构

场景	数据结构	示例
缓存对象详情	Hash（字段分组）	用户信息存储为 `user:1001` 的 Hash
排行榜/排序	ZSet（有序集合）	游戏积分排行榜 `leaderboard:game1`
社交关系（关注列表）	Set/Sorted Set	用户关注列表 `followers:1001`

（2）避免大 Value

单个 Value 过大（如超过 10KB）会阻塞 Redis 单线程，影响性能。

优化方案：

分片存储：将大 Hash 拆分为多个小 Hash（如 user:1001:info1, user:1001:info2）。

压缩数据：对 JSON/文本使用 GZIP 压缩（需权衡 CPU 开销）。

（3）合理设置过期时间（TTL）

缓存类数据必须设置 TTL，避免冷数据长期占用内存。

使用 EXPIRE 或 SETEX 命令，结合业务场景选择固定或随机过期时间（防缓存雪崩）。

案例：缓存场景（如用户信息缓存）

Key 设计如下：

```
user:profile:{user_id}    # 用户基础信息
user:perms:{user_id}      # 用户权限信息
```

Value 设计

```
SET user:profile:1001 '{"name":"Alice","age":30}'
```

面试篇之场景题

18、Spring AOP 在什么场景下会失效?

【小米】

以下是常见的失效场景及其原因与解决方案：

1. 内部方法调用失效

场景与原因

当一个 Bean 的内部方法直接调用另一个方法时，AOP 无法拦截。例如：

```
@Component
public class MyBean {
    public void methodA() {
        methodB(); // 直接内部调用，不走代理
    }
    @Transactional
    public void methodB() { ... }
}
```

原因：AOP 通过代理对象增强方法，但内部调用通过 this 直接调用目标对象的方法，绕过了代理。

解决方案：使用 AopContext.currentProxy()：需开启 exposeProxy 配置：

```
@EnableAspectJAutoProxy(exposeProxy = true)
public class AppConfig { ... }
// 调用方式
((MyBean) AopContext.currentProxy()).methodB();
```

2. 非 Spring 管理的对象

通过 new 关键字直接创建对象，而非通过 Spring 容器获取 Bean：

```
MyBean bean = new MyBean(); // 未被 Spring 管理
```

原因：AOP 只能代理由 Spring 容器管理的 Bean。

解决方案

使用 @Component 等注解将类声明为 Bean，并通过 @Autowired 注入。

3. 异步方法（@Async）

异步方法在新线程中执行时，AOP 拦截器可能无法跟踪：

```
@Async
public void asyncMethod() { ... }
```

原因：异步线程上下文与代理对象分离，导致切面逻辑失效。

解决方案

在异步方法外层调用处添加 AOP 逻辑，或使用线程本地变量传递上下文。

4.切入点表达式错误

表达式未正确匹配目标方法：

```
@Before("execution(* com.example.*.get*(..))") // 可能未覆盖实际方法
```

解决方案

使用精确的表达式，如 execution(* com.example.service.UserService.*(..))。

面试篇之场景题

19、接口防刷怎么实现?

【小米】

首先想到的是限流，比如使用令牌桶或漏桶算法，限制每个用户或 IP 的请求频率。然后可能需要验证码，特别是对于登录、注册等敏感接口。还有黑名单机制，自动封禁频繁违规的 IP 或用户账号。

另外，设备指纹和用户行为分析也是重要的一部分，可以通过收集客户端信息来识别唯一设备，分析请求模式是否异常。比如，正常用户不会在短时间内发起大量相同请求。

（1）IP 限流

分布式限流：使用 Redis + Lua 脚本实现原子化计数器（如令牌桶算法）。

```lua
-- KEYS[1]=限流Key（如ip:127.0.0.1），ARGV[1]=时间窗口（秒），ARGV[2]=最大请求数
local key = KEYS[1]
local limit = tonumber(ARGV[2])
local window = tonumber(ARGV[1])
local current = redis.call('GET', key) or 0
if tonumber(current) >= limit then
    return 0  -- 触发限流
else
    redis.call('INCR', key)
    redis.call('EXPIRE', key, window)
    return 1  -- 允许通过
end
```

```java
String key = "rate_limit:ip:" + clientIP;
boolean allowed = redisEval(luaScript, Collections.singletonList(key), "60", "100");
```

（2）设备指纹识别

```java
// 生成设备指纹（示例）
public String generateDeviceFingerprint(HttpServletRequest request) {
    String userAgent = request.getHeader("User-Agent");
    String ip = request.getRemoteAddr();
    String acceptLanguage = request.getHeader("Accept-Language");
    return DigestUtils.md5Hex(ip + userAgent + acceptLanguage);  // 简单哈希
}
```

更进一步，可以客户端埋点采集硬件参数（如屏幕分辨率、CPU 型号）。

（3）请求签名

客户端生成请求参数的有序拼接字符串。

使用 HMAC-SHA256 和密钥生成签名。

将签名附加到请求头

```java
// 客户端生成签名
String data = "param1=value1&param2=value2&timestamp=1620000000";
String signature = HmacUtils.hmacSha256Hex(apiKeySecret, data);

// 服务端验证
String serverSign = HmacUtils.hmacSha256Hex(apiKeySecret, data);
if (!serverSign.equals(clientSign)) {
    throw new ApiSecurityException("签名无效");
}
```

（4）添加黑名单

自建黑名单库，自动封禁频繁触发规则的设备/IP。

面试篇之场景题

20、JVM 中一次完整的 GC 流程（从 ygc 到 fgc）是怎样的

【百度】

我们先讲解下内存区域划分

首先，JVM 堆内存分为以下区域：

（1）年轻代（Young Generation）

Eden 区：对象初次分配的位置。

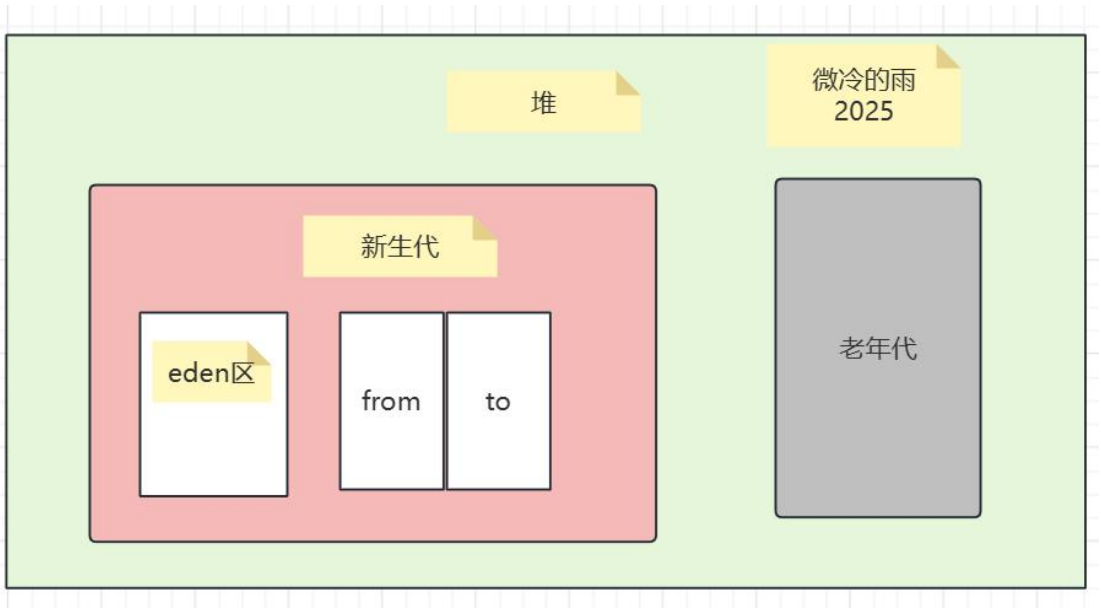
Survivor 区（S0 和 S1）：存放经过 Young GC 后存活的对象。

（2）老年代（Old Generation）

存放长期存活的对象（如缓存、全局变量）。

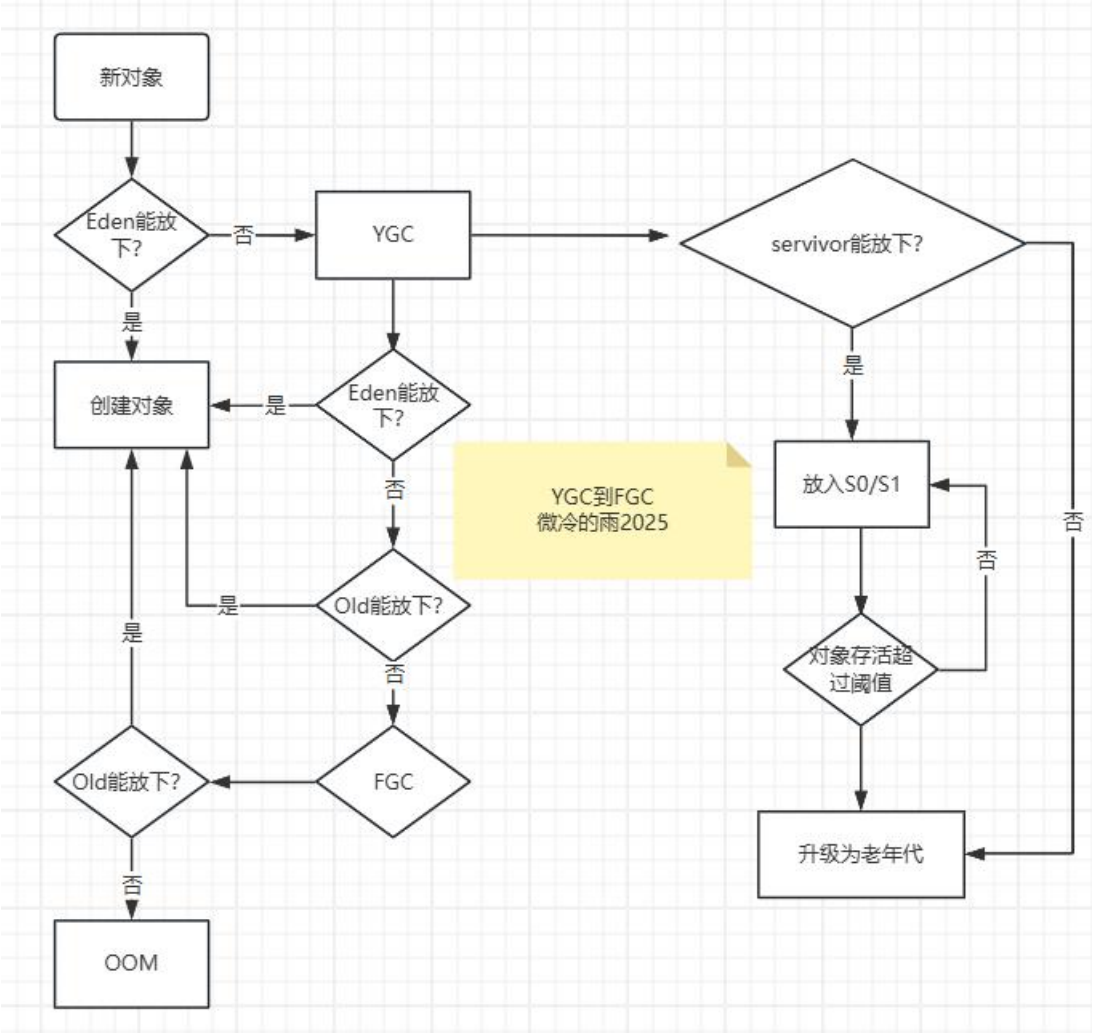
（3）元空间（Metaspace）

存储类元数据（JDK8+ 替代永久代）。



JVM 中一次完整的垃圾回收（GC）流程，通常包括两个部分：年轻代 GC 和老年代 GC。

其中年轻代 GC 又称为 YGC（Young Generation Garbage Collection），老年代 GC 又称 Major GC（Old GC）。



在年轻代中，对象创建主要发生在 Eden 区。然而，如果 Eden 区内存不足，可能会触发一次 Young GC。年轻代采用标记复制算法，将 Eden 区和 Survivor 区的内容复制到另一个 Survivor 区，最后清理 Eden 和 From Survivor 区的对象。

如果在这个过程中 Survivor 区无法容纳所有存活对象，就会进行空间分配担保。如果担保成功，则正常进行 Young GC；如果失败，可能需要触发 Full GC。如果老年代内存不足，则会触发老年代的 GC。在 Full GC 之后，如果老年代空间仍然不足，则会触发 OOM（Out Of Memory）错误。

总结来说，一次完整的 GC 过程包括 Young GC、Survivor 区复制、空间分配担保、Full GC 和老年

代 GC 等多个步骤，确保 JVM 内存管理的顺利进行。

面试篇之场景题

21、Spring 是如何管理事务的，事务管理机制?

【小米】

spring 事务核心组件有以下三个：

（1）PlatformTransactionManager

事务管理器接口，不同持久化技术有不同实现：

DataSourceTransactionManager：JDBC / MyBatis

HibernateTransactionManager：Hibernate

JpaTransactionManager：JPA

（2）TransactionDefinition

定义事务属性：隔离级别、传播行为、超时时间、只

读标志。

（3）TransactionStatus

事务运行时状态：是否是新事务、是否有保存点等。

那么 spring 是如何管理事务的呢?

1、Spring 通过 AOP 为 @Transactional 注解的方法生成代理对象。当调用代理对象的方法时：

（1）开启事务：

调用 TransactionManager.beginTransaction()

（2）执行业务逻辑

（3）提交事务：如果无异常

（4）回滚事务：如果抛出指定异常

2、在 @Transactional 中可以设置事务传播行为（Propagation）

REQUIRED（默认）：如果当前存在事务，则加入该事务；如果不存在，则新建事务。

REQUIRES_NEW：始终新建一个独立的事务，原事务被挂起。

NESTED：嵌套事务（基于保存点 Savepoint），子事务回滚不影响父事务。

3、也可以在 @Transactional 中设置事务隔离级别（Isolation）

READ_COMMITTED（默认）：避免脏读，但允许不可重复读和幻读。

REPEATABLE_READ：避免脏读和不可重复读。

SERIALIZABLE：最高隔离级别，避免所有并发问题。

数据表

```sql
-- 账户表
CREATE TABLE account (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id VARCHAR(50) NOT NULL,
    balance DECIMAL(10, 2) NOT NULL DEFAULT 0.00
);

-- 初始化数据
INSERT INTO account (user_id, balance) VALUES ('user1', 1000.00), ('user2', 500.00);
```

Maven 依赖

```
<dependencies>
    <!-- Spring Boot Starter JDBC -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-jdbc</artifactId>
        <version>2.7.0</version>
    </dependency>

    <!-- MySQL 驱动 -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
```

```
        <version>8.0.30</version>
    </dependency>

    <!-- Spring Boot Starter Test -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <version>2.7.0</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

SpringBoot 配置类

```java
@Configuration
@EnableTransactionManagement
@ComponentScan(basePackages = "cn.happy.myspringboot.day21.service")
// 确保扫描到 Service 类所在的包
public class AppConfig {

    // 数据源配置（需替换为你的数据库信息）
    @Bean
    public DataSource dataSource() {
        HikariDataSource dataSource = new HikariDataSource();
        dataSource.setJdbcUrl("jdbc:mysql://localhost:3306/book?useSSL=false");
        dataSource.setUsername("root");
        dataSource.setPassword("666666");
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        return dataSource;
    }
```

```
    // 显式注册 JdbcTemplate Bean
    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }

    // 事务管理器
    @Bean
    public PlatformTransactionManager transactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }
}
```

Service 层代码

```
@Service
public class AccountService {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    // 转账方法（声明式事务）
    2 usages
    @Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.READ_COMMITTED,
        rollbackFor = {SQLException.class, RuntimeException.class}
    )
```

```
    public void transferMoney(String fromUserId, String toUserId, BigDecimal amount) {
        // 1. 扣款
        jdbcTemplate.update(
            sql: "UPDATE account SET balance = balance - ? WHERE user_id = ?",
            amount, fromUserId
        );

        // 模拟异常（事务回滚）
        if (amount.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException("转账金额不能为负数");
        }

        // 2. 收款
        jdbcTemplate.update(
            sql: "UPDATE account SET balance = balance + ? WHERE user_id = ?",
            amount, toUserId
        );
    }
}
```

测试类代码

```java
@SpringBootTest(classes = AppConfig.class) // 显式指定配置类
public class AccountServiceTest {

    @Autowired
    private AccountService accountService;

    @Autowired
    private JdbcTemplate jdbcTemplate;
```

```java
    @Test
    public void testTransferSuccess() {
        // 执行转账（100元）
        accountService.transferMoney( fromUserId: "user1",  toUserId: "user2", new BigDecimal( val: "100.00"));

        // 验证余额变化
        BigDecimal user1Balance = jdbcTemplate.queryForObject(
                sql: "SELECT balance FROM account WHERE user_id = 'user1'", BigDecimal.class
        );
        BigDecimal user2Balance = jdbcTemplate.queryForObject(
                sql: "SELECT balance FROM account WHERE user_id = 'user2'", BigDecimal.class
        );
        assertEquals(new BigDecimal( val: "900.00"), user1Balance);
        assertEquals(new BigDecimal( val: "600.00"), user2Balance);
    }
```

```java
    @Test
    public void testTransferRollback() {
        // 转账金额为负数，预期触发回滚
        assertThrows(IllegalArgumentException.class, () -> {
            accountService.transferMoney( fromUserId: "user1",  toUserId: "user2", new BigDecimal( val: "-50.00"));
        });

        // 验证余额未变化
        BigDecimal balance = jdbcTemplate.queryForObject(
                sql: "SELECT balance FROM account WHERE user_id = 'user1'", BigDecimal.class
        );
        assertEquals(new BigDecimal( val: "1000.00"), balance);
    }
}
```

事务管理机制的核心是 @Transactional 注解与 PlatformTransactionManager 的协作，通过 AOP 动态代理实现事务的开启、提交和回滚。

面试篇之场景题

22、Mysql 为什么一定要有一个主键?

【小米】

首先，MySQL 中并没有强制要求表必须显式定义主键，但强烈建议为表设计主键。这种建议主要源于以下几个方面：

（1）存储引擎底层实现

1、InnoDB 的聚簇索引机制

InnoDB（MySQL 默认存储引擎）使用主键作为聚簇索引的组织依据，

数据按主键顺序物理存储。若未显式定义主键，InnoDB 会按以下顺序选择：

01、第一个非空的唯一索引（UNIQUE KEY）作为聚簇索引

02、自动生成 6 字节的隐藏 ROW_ID 作为伪主键

但是隐藏 ROW_ID 会导致问题：

所有无主键表共享全局序列，可能引发热点争用

无法利用主键查询的物理顺序性优势

2、二级索引的依赖

InnoDB 的二级索引叶子节点存储的是主键值（而非物理地址），无主键时，二级索引将存储隐藏的`ROW_ID`，导致：

索引占用空间增大

回表查询效率降低

（2）查询性能优化

1、快速数据定位

主键查找是最高效的单行访问方式（O(1)时间复杂度）

范围查询可利用主键顺序性减少磁盘 I/O

2、避免全表扫描

没有主键或合适索引时，简单查询都可能触发全表扫描，主键的自动索引特性天然避免这类问题。

（3）数据唯一性保证

1、实体完整性约束

主键的 NOT NULL + UNIQUE 特性确保每行数据可唯一标识。防止数据重复插入（业务层面无需额外校验）

2、外键关联基础

主外键关系是关系数据库的基石，无主键表无法被其他表有效引用

（4）运维与扩展需求

1、数据复制场景

主从复制（尤其 ROW 格式）中：

无主键表在备库回放时可能执行全表扫描

主键可显著提升复制效率（如并行复制基于主键分片）

分库分表设计

分布式 ID 生成依赖主键机制

数据路由通常基于主键哈希或范围划分

ORM 框架适配

大多数 ORM 工具要求表必须有主键

缺乏主键会导致对象关系映射困难

（5）设计规范角度

1、数据模型可读性

显式主键提升表结构的自解释性

符合数据库第三范式（3NF）设计要求

2、未来可扩展性

避免后期因业务变化临时添加主键的兼容性问题

主键数据类型选择影响存储效率（建议用自增整型）

最佳实践：

总是显式定义主键（建议 BIGINT， AUTO_INCREMENT）

避免使用业务字段作主键（除非符合自然主键条件）

复合主键仅在多对多关联表等特殊场景使用

面试篇之场景题

23、 如果 JVM 出现频繁 FullGC 该如何解决

【京东】

当 JVM 出现频繁 Full GC 时，通常表明内存管理存在严重问题，可能导致应用性能骤降甚至 OOM。

1、内存泄漏（最常见场景）

案例场景：

某电商平台的订单服务在促销期间频繁 Full GC，监控发现老年代内存使用率持续增长直至触发 Full GC，但 GC 后内存并未显著释放。

排查步骤：

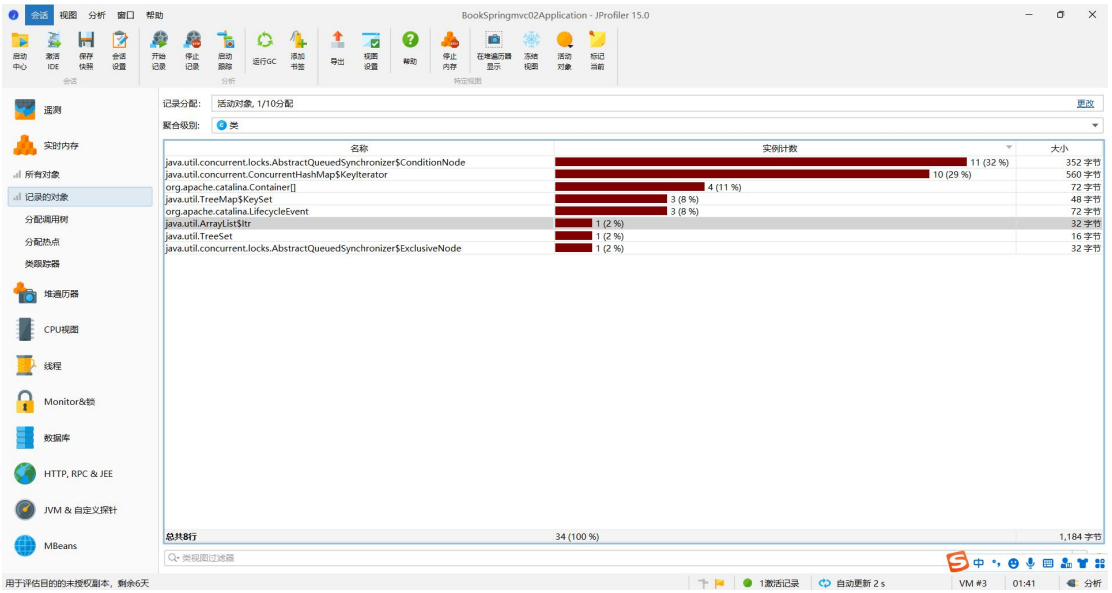
堆转储分析：使用 jmap -dump 生成堆快照，通过 JProfiler 分析对象分布。

jmap -dump:live,format=b,file=heap_dump.hprof <pid>

发现线索：发现 ConcurrentHashMap 中缓存了数千万个订单对象，且缓存未设置过期策略。

根因定位：代码中一个静态缓存 public static Map<Long, Order> orderCache 被误用作临时缓存，订单处理完成后未清理。

改用 WeakHashMap 设置合理的过期时间和最大容量解决问题。



2. 年轻代过小导致对象过早晋升

案例场景：

某数据分析服务在每天凌晨处理任务时频繁 Full GC，GC 日志显示每次 Minor GC 后存活对象较多，快速占满老年代。

排查步骤：

GC 日志分析：通过 -XX:+PrintGCDetails 发现 Survivor 区利用率 100%，对象年龄增长快。

-Xloggc:/gc.log -XX:+PrintGCDetails -XX:+PrintGCDateStamps

JVM 参数检查：发现 -Xmn200m（年轻代仅 200MB），而任务中单次批量处理的数据量达到 150MB。

解决方案：

01、调整分代比例：扩大年轻代（如 -Xmn2g），确保 Minor GC 后存活对象能放入 Survivor 区。

02、优化晋升阈值：调整 -XX:MaxTenuringThreshold=15，增加对象在年轻代的存活次数。

3、大对象直接分配至老年代

案例场景：

某图像处理服务在解析高分辨率图片时频繁 Full GC，日志显示每次 Full GC 前老年代突然被占满。

排查步骤：

代码审查：发现代码中通过 byte[] buffer = new byte[10 * 1024 * 1024] 分配 10MB 大数组。

JVM 机制：因未设置 -XX:PretenureSizeThreshold，默认阈值较低，大对象直接进入老年代。

解决方案：

优化内存分配：改用分块处理（如按 1MB 分块读取），避免单次分配大对象。

调整 JVM 参数：显式设置 -XX:PretenureSizeThreshold=20M，允许 20MB 以下对象在年轻代分配。

4. 元空间（Metaspace）不足

案例场景：

某微服务应用动态生成代理类（如 Spring AOP），运行一段时间后频繁 Full GC，日志显示 Metaspace 持续增长。

排查步骤：

监控元空间：使用 jstat -gc <pid> 观察 MC（Metaspace Capacity）和 MU（Metaspace Usage）。

发现异常：MU 接近 MaxMetaspaceSize（默认无限制，但受物理内存约束）。

解决方案：

限制元空间：设置 -XX:MaxMetaspaceSize=256M，避免元空间无限膨胀。

排查类加载器泄漏：发现某第三方库未正确关闭类加载器，修复后 Metaspace 使用稳定。

5. 外部资源阻塞导致对象堆积

案例场景：

某支付系统调用外部银行接口超时，线程阻塞导致任务队列堆积，短时间内产生大量未处理的 PaymentRequest 对象。

排查步骤：

线程分析：通过 jstack <pid> 发现大量线程处于 BLOCKED 状态，等待 I/O。

堆分析：未完成的 PaymentRequest 因线程未释放而无法回收，占满老年代。

解决方案：

设置超时与熔断：在 HTTP 调用中增加 connectTimeout=3000ms，并引入熔断器（如 Hystrix）。

优化线程池：使用有界队列 new ArrayBlockingQueue(1000) 并设置拒绝策略，避免任务无限堆积。

面试篇之场景题

24、JVM OOM 问题如何排查和解决

【京东】

一、确认 OOM 类型

java.lang.OutOfMemoryError: Java heap space（堆内存不足）

java.lang.OutOfMemoryError: Metaspace（元数据区溢出）

java.lang.OutOfMemoryError: Direct buffer memory（直接内存溢出）

java.lang.OutOfMemoryError: Unable to create new native thread（线程数过多）

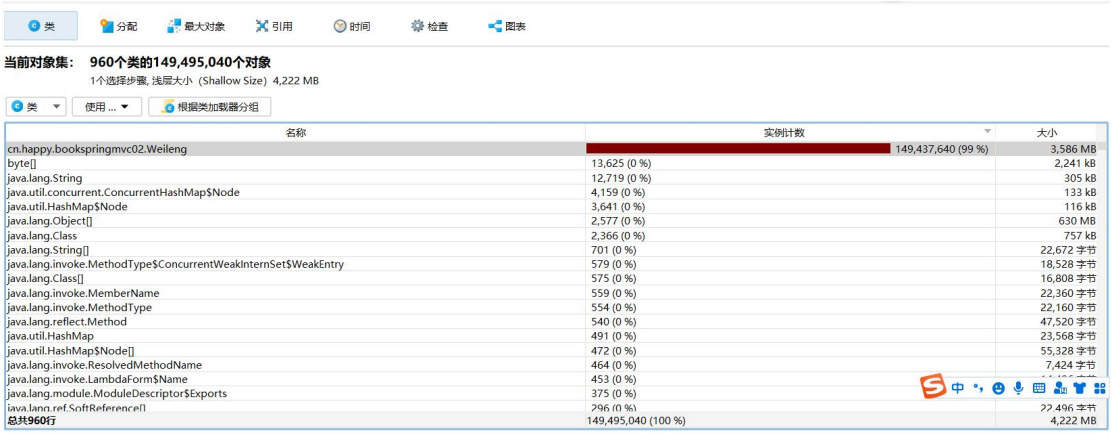
二、堆内存 OOM 排查（Java heap space）

1. 获取堆内存快照（Heap Dump）

启动时添加 JVM 参数 -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/path/to/dump，OOM 时自动生成 Dump 文件。

2. 分析堆内存快照

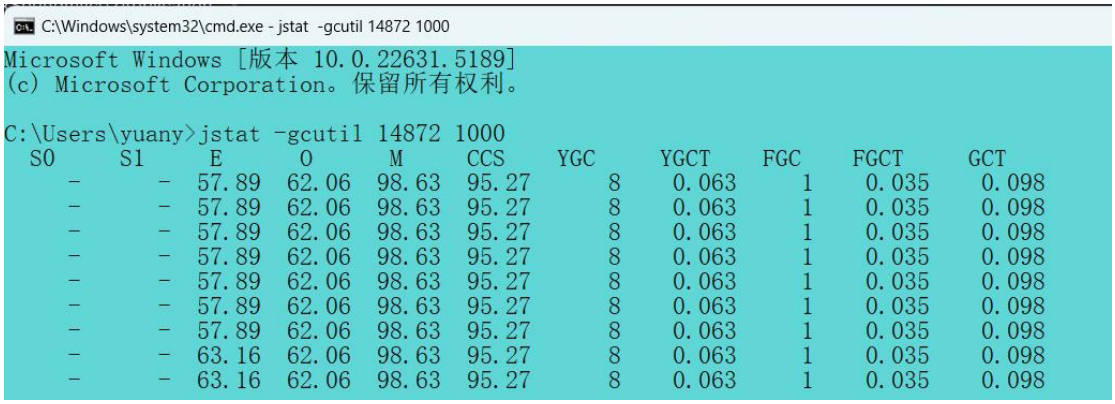
我们可以使用 JProfiler。查找占用内存最大的对象（Dominator Tree）。检查对象引用链（Path to GC Roots），识别是否因代码逻辑（如静态集合缓存未清理）导致内存泄漏。定位重复创建的大对象（如大数组、缓存数据）。



3. 监控 GC 行为

命令：jstat -gcutil <pid> 1000 观察 GC 频率和内存回收效果。

频繁 Full GC 且内存回收率低，可能为内存泄漏。结合第三方工具 gceasy 分析 GC 日志。



S0：幸存 1 区当前使用比例

S1：幸存 2 区当前使用比例

E：伊甸园区使用比例

O：老年代使用比例

M：元数据区使用比例

CCS：压缩使用比例

YGC：年轻代垃圾回收次数

FGC：老年代垃圾回收次数

FGCT：老年代垃圾回收消耗时间

GCT：垃圾回收消耗总时间

4. 检查 JVM 参数配置

确认堆大小（-Xms、-Xmx）是否合理，是否过小或未按负载调整。

检查新生代/老年代比例（-XX:NewRatio）是否合理，避免短生命周期对象晋升过快。

三、非堆内存 OOM 排查

1. Metaspace 溢出

原因：动态生成类（如反射、CGLIB 代理）过多，或元空间配置过小。

解决：

调整参数：-XX:MetaspaceSize、-XX:MaxMetaspaceSize。

检查代码，避免重复生成代理类。

2. 线程数溢出

解决：

调整系统级限制（ulimit -u）。

优化线程池配置，避免无界线程池。

四、代码级优化

1. 内存泄漏修复

静态集合缓存未清理 → 改用弱引用（WeakHashMap）或添加过期策略。

资源未关闭（如数据库连接、文件流）

2. 减少内存占用

避免大对象频繁创建（如大数组、大字符串）→ 优化为分块处理或复用对象。

面试篇之场景题

25、CPU 使用率较高排查和解决

【京东】

一、快速定位问题进程和线程

确定高 CPU 进程

top -c # 按 CPU 排序，找到占用高的 Java 进程 PID

定位进程内高 CPU 线程

top -Hp <PID> # 查看线程级 CPU 占用，记录高占用的线程 ID（十进制）

二、分析线程堆栈定位代码问题

生成线程转储（Thread Dump）文件

jstack <PID> > thread_dump.txt

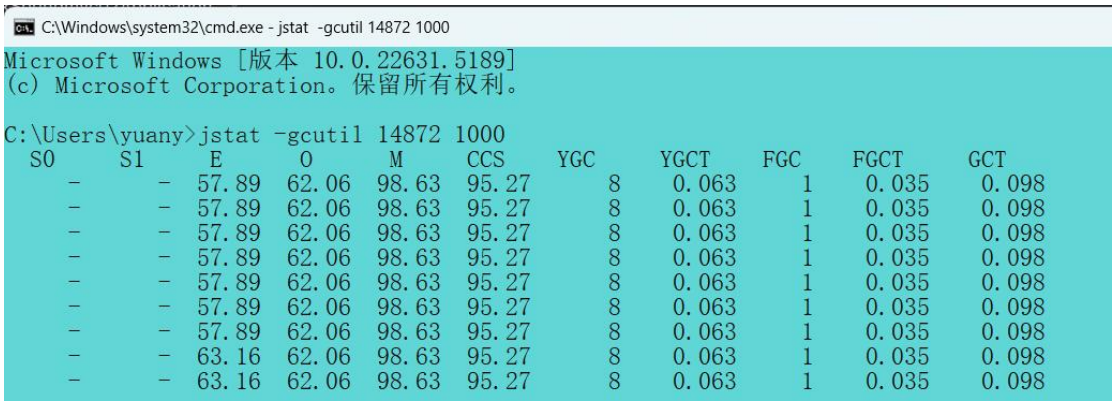
结合十六进制线程 ID，在 thread_dump.txt 中查找对应线程的堆栈信息。在输出中寻找处于 RUNNABLE 状态的线程，分析它们的调用栈，找到导致高 CPU 的原因。

三、检查 GC 活动

关注 FGC（Full GC 次数）和 FGCT（Full GC 耗时），频繁 Full GC

可能导致 CPU 飙升。

jstat -gcutil <PID> 1000 # 每秒输出一次 GC 统计



四、代码和配置优化

1、代码层优化

优化算法或循环逻辑（如减少不必要的计算、使用缓存）。

避免锁竞争：改用并发容器（如 ConcurrentHashMap）、减小锁粒度或使用无锁结构。

2、JVM 调优

调整堆大小：-Xms 和-Xmx 避免频繁扩容。

选择低停顿 GC 器：如 G1 或 ZGC（-XX:+UseG1GC）。

优化元空间：避免类加载导致 Metaspace OOM（-XX:MaxMetaspaceSize）。

3、配置检查

调整线程池大小，避免过多线程竞争 CPU。

关闭不必要的日志输出（如 DEBUG 级别）。

五、系统级排查

监控网络/磁盘 IO，高 IO 等待可能间接导致 CPU 负载上升。

六、长期预防措施

监控告警：设置阈值告警（如 CPU 持续>80%触发通知）。

压测与优化：定期进行压力测试，提前发现性能瓶颈。

面试篇之场景题

26、Mysql 中的 RR 隔离级别，到底有没有解决幻读问题

【腾讯】

MySQL 的 InnoDB 引擎在可重复读（RR）隔离级别下，通过多版本并发控制（MVCC）和间隙锁（Gap Lock）的结合，基本解决了幻读问题，但其解决方式依赖于具体操作类型和锁机制的使用。

1. 快照读（Snapshot Read）与幻读

普通 SELECT 操作（非锁定读）使用一致性视图（基于事务开始时的快照），确保多次读取结果一致，因此不会看到其他事务插入的新数据，避免了幻读。

```
-- 事务A
BEGIN;
SELECT * FROM bookinfo WHERE price > 100; -- 快照读，返回空
-- 事务B插入age=25的记录并提交
SELECT * FROM bookinfo WHERE price > 100; -- 仍返回空（无幻读）
COMMIT;
```

2. 当前读（Current Read）与幻读

锁定读（SELECT ... FOR UPDATE）、UPDATE/DELETE 操作属于当前读，会读取最新已提交数据并加锁：间隙锁会锁定索引范围的间隙，阻止其他事务在范围内插入，从而避免幻读。

```
-- 事务A
BEGIN;
SELECT * FROM users WHERE age > 20 FOR UPDATE; -- 加间隙锁，阻止插入
-- 事务B尝试插入age=25的记录会被阻塞
COMMIT;
```

3. 混合操作的潜在问题

如果事务中混合快照读与当前读，可能出现数据版本不一致：

```
-- 事务A
BEGIN;
SELECT * FROM users WHERE age > 20; -- 快照读，返回空
-- 事务B插入age=25并提交
UPDATE users SET status='active' WHERE age > 20; -- 当前读，发现新数据并更新
SELECT * FROM users WHERE age > 20; -- 当前读后，后续快照读可能看到新数据（幻读）
COMMIT;
```

UPDATE 操作会更新快照，导致后续查询基于新版本数据，可能暴露幻读。但此现象实际是 InnoDB 的当前读机制导致，而非严格意义上的幻读（因事务内数据逻辑一致性仍被维护）。

因此，虽然 RR 隔离级别通过间隙锁机制在很大程度上防止了幻读，但在某些特定的读取方式和操作下，幻读仍然可能发生。要完全避免幻读，可以使用 Serializable 隔离级别，但这通常会降低系统的并发性能。

面试篇之场景题

27、Mysql 的行级锁到底锁的是什么东西?

【小米】

MySQL 的行级锁在 InnoDB 存储引擎中通过锁定索引记录来实现，其核心是锁住索引项而非物理数据行。以下从多个维度全面解析行级锁的机制：

1. 行级锁的实现基础：索引

索引是锁的载体：InnoDB 的行级锁作用于索引记录。若查询通过索引（主键或二级索引）定位数据，锁会加在对应的索引项上。

无索引时的行为：若查询未使用索引，InnoDB 会退化为全表扫描，逐行加锁（实际通过隐藏的聚簇索引），此时可能等效于锁表，导致性能下降。

2. 行级锁的类型

InnoDB 的行级锁分为三种，具体行为受事务隔离级别影响：

锁类型	描述	隔离级别
记录锁（Record Locks）	锁定索引中的单条记录（如主键或唯一索引），阻止其他事务修改或删除该行。	所有隔离级别均使用。
间隙锁（Gap Locks）	锁定索引记录之间的间隙(如 id=5 和 id=10 之间的区间），防止插入新数据，解决幻读	仅在可重复读（RR）下生效。

临键锁（Next-Key Locks）	记录锁 + 间隙锁的组合，锁定一个左开右闭的索引区间（如(5,10]）。	默认用于 RR 隔离级别。

3. 锁的触发场景

显式加锁：通过 SELECT ... FOR UPDATE（X 锁）或 SELECT ... FOR SHARE（S 锁）手动加锁。

隐式加锁：DML 语句（如 UPDATE、DELETE、INSERT）自动为相关行加排他锁（X 锁）。

4. 锁的释放时机

事务结束时释放：通过 COMMIT 或 ROLLBACK 释放锁。

长时间持有风险：未提交的事务可能导致锁堆积，引发死锁或性能问题。

5. 二级索引与主键索引的联动

二级索引加锁时：除锁定二级索引项外，还会回溯到主键索引并加锁（防止其他事务通过主键修改数据）。

```
-- 假设表 t 有主键`id`和二级索引`age`
UPDATE t SET name = 'Alice' WHERE age = 20;
```

锁定 age=20 的二级索引项。

根据二级索引项找到对应的主键值，锁定主键索引项。

6. 不同隔离级别下的锁策略

隔离级别	锁策略
读未提交（RU）	不加锁（实际通过 MVCC 读取最新数据，可能脏读）。
读已提交（RC）	仅加记录锁，无间隙锁（可能幻读）。

可重复读（RR）	默认使用临键锁，防止幻读（通过间隙锁和记录锁组合）。
串行化（Serializable）	所有 SELECT 隐式转为 FOR SHARE，严格串行。

7. 死锁与检测机制

死锁成因：多个事务以不同顺序请求锁，形成循环等待。

解决方案：InnoDB 自动检测死锁，强制回滚代价较小的事务，并抛出 1213 错误。

MySQL 的行级锁通过索引项实现，锁定的本质是索引记录，而非物理数据行。

面试篇之场景题

28、存储 MD5 值应该用 VARCHAR 还是用 CHAR?

【京东】

1、先说结论，存储 MD5 值建议使用 CHAR(32) 类型，而非 VARCHAR。

首先，我们先讲解下 MD5 的特征，MD5 哈希值是一个 固定长度（32 位） 的十六进制字符串，由 0-9 和 a-f （或 A-F ） 组成，例如： 1 的 MD5 值为 C4CA4238A0B923820DCC509A6F75849B。无论输入数据如何变化，MD5 的输出长度始终固定为 32 个 字符，没有变长需求。

2、接下来讲解下 CHAR 和 VARCHAR 的核心区别：

CHAR(n)：固定长度，占用空间始终为 n 字符（例如 CHAR(32) 严格占用 32 字符的存储空间）。适合存储长度严格固定的数据（如 MD5、UUID、加密盐值等）。读写效率通常略高于 VARCHAR，因为无需计算实际长度。

VARCHAR(n)：可变长度，占用空间为 实际字符数 + 长度标识（例如 VARCHAR(32) 需要额外 1-2 字节存储长度信息）。适合存储长度可能变化的数据（如用户名、地址等）。对于固定长度的数据，VARCHAR 会浪费存储空间。

3、性能与存储优化

查询效率

固定长度的 CHAR 字段在索引、排序、哈希计算等操作中更高效，因为数据库引擎可以直接按固定长度处理，无需解析动态长度。

存储引擎适配性

对于 MySQL 的 InnoDB 引擎，固定长度的字段更容易对齐内存页，减少碎片化。

面试篇之场景题

29、聊聊你印象最深刻的项目，或者做了什么优化。

【京东】

我参与过一个千万级用户量的电商平台核心系统重构，其中负责优化订单查询模块。原系统在高并发场景下（如大促期间），订单列表页加载延迟高达 5-8 秒，用户投诉率上升，

直接影响转化率。

一、问题分析

数据库瓶颈

订单表数据量超 2 亿条，查询时频繁全表扫描（如按用户 ID+时间范围查询）。

索引设计不合理：仅对用户 ID 建立了单字段索引，未覆盖时间范围查询，导致大量回表操作。

代码逻辑缺陷

分页查询使用 OFFSET 10000 LIMIT 20，深度分页效率极低。

冗余字段加载：ORM 框架默认全字段查询，但列表页仅需展示订单号、金额等核心字段。

缓存策略缺失

高频访问的“最近 3 个月订单”未做缓存，重复访问直接穿透数据库。

二、优化方案与落地

1. 数据库优化

索引重构：

将用户 ID 单列索引改为联合索引 (user_id, create_time)，覆盖高频查询条件，减少回表次数。

引入时间分区表（按月分区），加速时间范围扫描。

分页优化：

将 OFFSET 改为游标分页（基于 create_time 和 order_id），避免深分页性能问题。

字段精简：

重写 SQL 语句，仅查询 SELECT order_id, amount, status, create_time 等必要字段，减少数据传输量。

2. 缓存设计

多级缓存策略：

一级缓存（本地缓存）：存储用户最近 10 条订单（TTL=10 分钟），应对瞬时高频请求。

二级缓存（分布式缓存）：Redis 存储用户近 3 个月订单，使用 Hash 结构按用户 ID 分组，设置动态 TTL（最近访问用户延长缓存时间）。

缓存预热：

每日凌晨低峰期，通过定时任务预加载活跃用户的最近订单数据。

3. 代码层优化

异步化处理：

将非核心逻辑（如订单状态变更日志记录）迁移到消息队列（Kafka），由消费者异步处理。

连接池调优：

调整 Druid 连接池参数，maxActive 从 50 提升至 200（基于压测结果），减少获取连接的等待时间。

三、优化效果

性能指标

订单查询平均响应时间从 5.8 秒降至 320 毫秒。

数据库 CPU 峰值负载从 90%降低至 35%，慢查询数量减少 98%。

业务收益

大促期间订单页崩溃率归零，转化率提升 12%。

服务器资源成本节省约 40%（缩减了 3 台数据库从节点）。

该项目的核心价值在于通过系统性分析瓶颈（数据库→代码→架构）、分层优化手段（存储、计算、缓存）与 数据驱动决策（监控+压测验证），最终实现性能质的飞跃。这也让我深刻体会到，高并发优化不仅是技术问题，更是对业务场景、用户习惯、资源成本的综合权衡。

面试篇之场景题

30、Mysql 是如何解决幻读问题的?

【阿里】

MySQL 通过 可重复读（Repeatable Read）隔离级别 下的 MVCC（多版本并发控制） 和 间隙锁（Gap Lock） 机制解决幻读问题。

一、幻读的定义与场景

幻读是指在同一事务中，多次执行相同范围的查询时，由于其他事务插入或删除符合该范围

的数据，导致前后结果集不一致的现象。

示例场景：

事务 A 查询年龄在 20~30 岁的用户，返回 5 条记录。

事务 B 插入一个年龄为 25 岁的新用户并提交。

事务 A 再次查询相同范围，发现返回 6 条记录（多了一条事务 B 插入的数据）。

二、MySQL 如何解决幻读?

1. MVCC（多版本并发控制）

原理：

每个事务启动时生成一个一致性视图（Read View），后续查询基于该视图读取数据快照，即使其他事务修改或插入数据，当前事务看到的仍是快照版本。

解决快照读的幻读：

对于普通 SELECT 语句（快照读），MVCC 保证事务内多次查询结果一致，不会看到其他事务插入的新数据。

```sql
-- 事务A（隔离级别：可重复读）
BEGIN;
SELECT * FROM users WHERE age BETWEEN 20 AND 30; -- 返回5条记录

-- 事务B插入数据并提交
INSERT INTO users (name, age) VALUES ('John', 25);
COMMIT;

-- 事务A再次查询（仍返回5条记录，无幻读）
SELECT * FROM users WHERE age BETWEEN 20 AND 30;
COMMIT;
```

结果：事务 A 两次查询结果一致，MVCC 通过快照隔离避免了幻读。

2. 间隙锁（Gap Lock）

原理：

针对当前读（如 SELECT ... FOR UPDATE 或 UPDATE/DELETE 语句），InnoDB 在索引记

录的间隙加锁，阻止其他事务在范围内插入新数据。

解决当前读的幻读：

锁定查询范围的“间隙”（即使该位置无实际数据），确保事务执行期间不会有新数据插入。

```sql
-- 事务A（隔离级别：可重复读）
BEGIN;
SELECT * FROM users WHERE age BETWEEN 20 AND 30 FOR UPDATE; -- 当前读，加间隙锁

-- 事务B尝试插入年龄25的用户（被阻塞）
INSERT INTO users (name, age) VALUES ('John', 25); -- 等待事务A提交或回滚

-- 事务A提交后，事务B的插入才能执行
COMMIT;
```

三、关键注意事项

1. 索引对间隙锁的影响

若查询条件无索引，InnoDB 会退化为表级锁，锁住整张表，严重影响并发性能。

最佳实践：为范围查询字段（如 age）建立索引，确保间隙锁精准锁定目标范围。

2. MVCC 与间隙锁的协作

快照读（SELECT）：依赖 MVCC，无锁机制，通过版本链实现一致性视图。

当前读（SELECT ... FOR UPDATE）：依赖间隙锁，物理阻塞其他事务插入。

3. 隔离级别的选择

仅在可重复读（默认隔离级别） 下，InnoDB 自动启用间隙锁。

若使用读已提交（Read Committed），间隙锁仅在“外键约束”和“唯一性检查”时生效，无法避免幻读。

四、实际应用案例

场景：电商系统订单状态批量更新

需求：将用户未支付的超时订单（如创建时间超过 30 分钟）批量关闭。

问题：在高并发场景下，若直接执行 UPDATE orders SET status='closed' WHERE status='unpaid' AND created_at < NOW() - 1800，可能因其他事务插入新订单导致幻

读。

```sql
BEGIN;
-- 当前读 + 间隙锁：锁定符合条件的记录及间隙
SELECT * FROM orders
WHERE status='unpaid' AND created_at < NOW() - 1800
FOR UPDATE;

-- 执行更新（间隙锁持续生效，阻止新订单插入范围）
UPDATE orders SET status='closed'
WHERE status='unpaid' AND created_at < NOW() - 1800;
COMMIT;
```

效果：通过 FOR UPDATE 触发的间隙锁，确保更新过程中不会有新订单插入，彻底避免幻读。

五、总结

MVCC：解决快照读的幻读，通过一致性视图隔离其他事务的修改。

间隙锁：解决当前读的幻读，物理阻塞范围内数据插入。

最佳实践：合理设计索引、根据业务需求选择隔离级别，结合 FOR UPDATE 显式控制锁粒度。

面试篇之场景题

31、请说一下对象的创建过程

【阿里】

1. 类加载检查

当 JVM 遇到 new 指令时，首先检查对应的类是否已经加载、解析、初始化。

若未加载：触发类加载过程（加载 → 验证 → 准备 → 解析 → 初始化）。

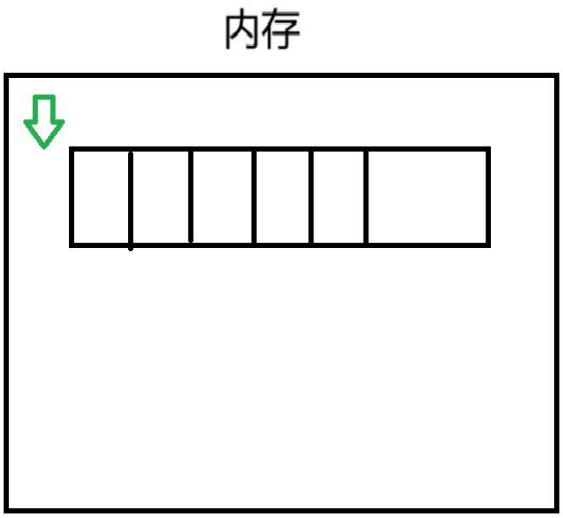
若已加载：直接进入内存分配阶段。

```
Object obj = new Object(); // 触发类加载检查
```

2. 内存分配

JVM 为对象分配内存空间（堆内存），分配方式取决于堆内存的规整性：

分配方式	适用场景	实现机制
指针碰撞（Bump the Pointer）	堆内存规整（如 Serial、ParNew 等垃圾收集器）	通过移动指针划分内存区域
空闲列表（Free List）	堆内存不规整（如 CMS 收集器）	维护空闲内存块列表，分配时查找可用空间



3. 初始化零值

内存分配完成后，JVM 将对象的所有实例变量初始化为零值（默认值）：

int 赋值为 0

boolean 赋值为 false

引用类型 赋值为 null



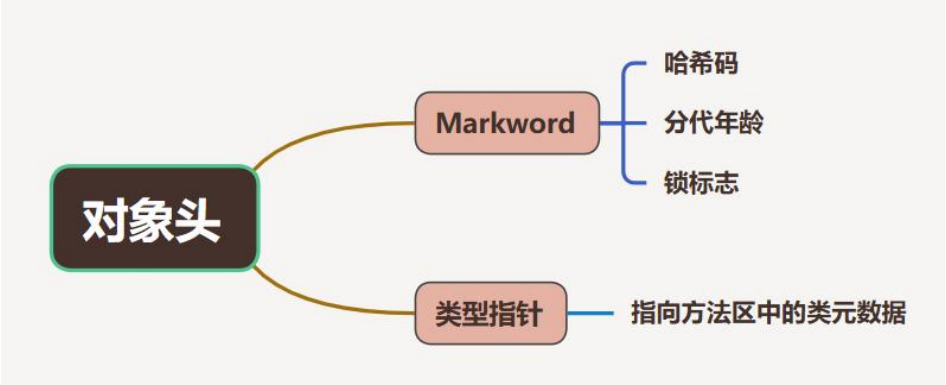
4. 设置对象头（Object Header）

JVM 在对象头中存储元数据信息，包括：

Mark Word：哈希码、GC 分代年龄、锁状态标志等。

类型指针：指向类元数据的指针（用于确定对象类型）。

对象头结构示意图：



5. 执行 <init> 方法（构造方法）

执行对象的构造函数（<init> 方法），完成以下操作：

调用父类构造函数（递归执行）。

显式初始化成员变量（如 int age = 18;）。

执行构造代码块和构造函数中的代码。

```
public class Person {
    private int age = 18;   // 显式初始化
    { age = 20; }           // 构造代码块
    public Person() {
        age = 25;           // 构造函数赋值
    }
}
// 最终 age = 25
```

内存变化：

初始化零值 → age=0

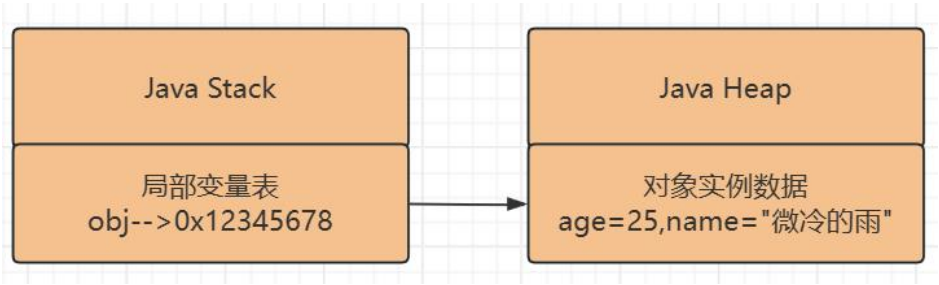
显式初始化 → age=18

构造代码块 → age=20

构造函数 → age=25

6. 对象引用关联

将堆内存中的对象地址赋值给栈中的引用变量，完成对象创建。



总结：

```
类加载检查
↓
内存分配（指针碰撞/空闲列表）
↓
初始化零值（默认值）
↓
设置对象头（Mark Word、类型指针）
↓
执行构造方法（父类构造 → 显式初始化 → 构造代码块 → 自定义逻辑）
↓
引用关联（栈变量指向堆对象）
```

面试篇之场景题

32、HashMap 是怎么解决哈希冲突的?

【腾讯】

哈希冲突（Hash Collision）是指不同的键通过哈希函数计算后，得到的哈希值（或数组索引）相同。由于 HashMap 使用有限的数组存储数据，而键的取值范围可能是无限的，冲突不可避免。

1. 链地址法（Separate Chaining）

基本原理：每个数组（桶）位置存储一个链表或红黑树。当多个键的哈希值映射到同一位置时，这些键值对以链表形式链接；当链表长度超过阈值时，转换为红黑树以提高查询效率。

数据结构：

数组 + 链表（默认）：哈希冲突的键值对以链表形式存储。

数组 + 红黑树（Java 8+）：链表长度超过阈值（默认为 8）时，自动转换为红黑树；节点数减少到阈值以下（默认为 6）时，恢复为链表。

2. 哈希函数优化

扰动函数：对键的 hashCode()进行二次处理，减少哈希冲突概率。

```
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

3. 动态扩容（Rehashing）

负载因子（Load Factor）：默认 0.75，当元素数量超过容量 × 负载因子时触发扩容。

扩容流程：

新建一个 2 倍大小的数组。

重新计算所有元素的哈希值并分配到新数组的对应位置。

迁移后旧数组被丢弃，新数组成为主存储结构。

效果：减少哈希冲突概率，提高查询效率。

4. 红黑树优化（Java 8+）

转换条件：

链表 → 红黑树：链表长度 ≥ 8 且数组容量 ≥ 64（否则优先扩容）。

红黑树 → 链表：节点数 ≤ 6。

优势：红黑树的查找时间复杂度为 O(log n)，显著优化长链表的性能。

流程示例

插入键值对：

计算键的哈希值，定位到数组索引。

若该位置为空，直接存入新节点。

若冲突，遍历链表/红黑树：

链表：插入到尾部（Java 7 头插，Java 8 尾插）。

红黑树：按树结构插入。

检查链表长度，决定是否转换结构。

查询键值对：

计算哈希值定位数组索引。

遍历链表或红黑树，通过 equals()匹配键。

```java
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 1);  // 哈希值计算后定位到索引i
map.put("B", 2);  // 假设与"A"哈希冲突，存入同一位置的链表
map.put("C", 3);  // 链表长度增加，超过阈值则转换为红黑树
```

对比不同版本的实现

版本	冲突处理	插入方式	扩容策略
Java 7	纯链表	头插法	先扩容后插入
Java 8+	链表 + 红黑树	尾插法	先插入后检查扩容

面试篇之场景题

33、什么是拆包和粘包？怎么解决？

【小米】

一、什么是拆包和粘包？

在 基于流的传输协议（如 TCP） 中，数据没有明确的边界标识，发送方的多次写入操作可能被接收方一次性读取，或一次写入的数据被接收方分多次读取。这种现象会导致以下两种问题：

粘包（Packet Sticking）

多个数据包被合并成一个大的数据块接收（例如：发送方连续发送 A|B，接收方可能一次性收到 AB）。

拆包（Packet Splitting）

一个数据包被拆分成多个小块接收（例如：发送方发送 BigData，接收方可能先收到 Bi，再收到 gData）。

二、问题根源

TCP 协议特性：TCP 是面向字节流的协议，保证数据可靠性和顺序性，但不维护数据边界。

缓冲区机制：发送方和接收方的缓冲区大小、网络状况（如 MTU 限制）可能导致数据分段或合并。

Nagle 算法：为提高网络效率，TCP 可能合并多个小数据包发送。

三、解决方案

以下是常见的解决方案，按实现复杂度排序：

1. 固定长度法（Fixed-Length）

核心思想：每个数据包长度固定，不足部分用填充字符补全。

实现方式：

```
// 发送方：固定每个包长度为 10 字节，不足补空格
byte[] data = "Hello".getBytes();
byte[] fixedData = Arrays.copyOf(data, 10);
Arrays.fill(fixedData, data.length, 10, (byte) ' ');
```

```
// 接收方：每次读取固定长度
ByteBuffer buffer = ByteBuffer.allocate(10);
while (buffer.remaining() > 0) {
    channel.read(buffer);
}
buffer.flip();
```

优点：实现简单，解码高效。

缺点：浪费带宽，不适合变长数据。

2. 分隔符法（Delimiter-Based）

核心思想：用特殊字符（如 \n）标记数据包结束。

实现方式（以 Netty 为例）：

```
// 添加行解码器（按 \n 或 \r\n 分割）
pipeline.addLast(new LineBasedFrameDecoder(1024));
pipeline.addLast(new StringDecoder());
```

关键点：

需对数据中的分隔符转义（如 HTTP 头中的 \r\n\r\n）。

不适合二进制数据（可能包含分隔符字节）。

优化方案：使用复杂分隔符（如 #$%^）降低冲突概率。

3. 长度字段法（Length Field-Based）

核心思想：在数据包头添加长度字段，明确标识包体长度。

协议格式：



实现步骤：

编码（发送方）：先写入数据长度（4 字节），再写入数据。

```
ByteBuf buffer = Unpooled.buffer();
byte[] data = "Hello".getBytes();
buffer.writeInt(data.length); // 写入长度字段
buffer.writeBytes(data);      // 写入实际数据
```

解码（接收方）：先读取长度字段，再按长度读取后续数据。

```
// Netty 内置 LengthFieldBasedFrameDecoder 自动处理
pipeline.addLast(new LengthFieldBasedFrameDecoder(1024, 0, 4));
```

4. 高级协议设计

HTTP/1.1：使用 Content-Length 或 Transfer-Encoding: chunked。

HTTP/2：通过二进制分帧层（Frame）划分数据边界。

自定义协议：结合长度字段、魔数（Magic Number）、校验码（CRC）等提高健壮性。

魔数	Length	CRC32	Data
0xFE	4 字节	4 字节	变长内容

五、总结

方案	适用场景	优点	缺点
固定长度	数据长度固定	简单高效	浪费带宽

分隔符	文本协议	直观易实现	需处理转义，不适合二进制数据
长度字段	二进制协议、高可靠性场景	灵活可靠，通用性强	需额外处理长度字段
高级协议设计	复杂业务场景（如 HTTP）	功能强大，扩展性好	实现复杂度高

最终建议：

生产环境中推荐使用长度字段法，结合 Netty 的 LengthFieldBasedFrameDecoder 可高效解决粘包/拆包问题。

面试篇之场景题

34、什么是 SPI，它有什么用?

【小米】

一、什么是 SPI?

SPI（Service Provider Interface）是 Java 提供的一种服务发现机制，允许框架或核心库动态加载第三方实现，而无需硬编码依赖。其核心思想是面向接口编程，通过配置文件实现接口与实现的解耦，常用于构建可扩展的插件化架构。

二、SPI 与 API 的区别

特性	API	SPI
面向对象	提供给调用方使用的接口（如 Java 类库）	提供给服务提供者实现的接口
关注点	如何调用功能	如何扩展功能
控制方向	由实现方定义接口，调用方直接使用	由调用方定义接口，实现方提供具体实现
典型场景	Java 标准库中的 List、Map	JDBC 驱动、日志框架桥接

三、SPI 的核心机制

1. 核心类：ServiceLoader

Java 通过 java.util.ServiceLoader 类实现 SPI 机制，其工作流程如下：

定位配置文件：在 JAR 包的 META-INF/services/ 目录下查找以接口全限定名命名的文件。

解析实现类：读取文件内容，获取实现类的全限定名。

实例化对象：通过反射（Class.forName()）加载并实例化实现类。

2. 配置文件格式

文件路径：META-INF/services/<接口全限定名>

文件内容：每行一个实现类的全限定名（支持多实现）。

```
com.example.MyServiceImpl1
com.example.MyServiceImpl2
```

四、SPI 的实战步骤

1. 定义服务接口

```
// 服务接口：数据加密
public interface Encryptor {
    String encrypt(String data);
}
```

2. 实现服务接口

```
// AES 加密实现
public class AesEncryptor implements Encryptor {
    @Override
    public String encrypt(String data) {
        return "AES_ENCRYPTED:" + data;
    }
}
```

```
// RSA 加密实现
public class RsaEncryptor implements Encryptor {
    @Override
    public String encrypt(String data) {
        return "RSA_ENCRYPTED:" + data;
    }
}
```

3. 创建 SPI 配置文件

文件路径：META-INF/services/com.example.Encryptor

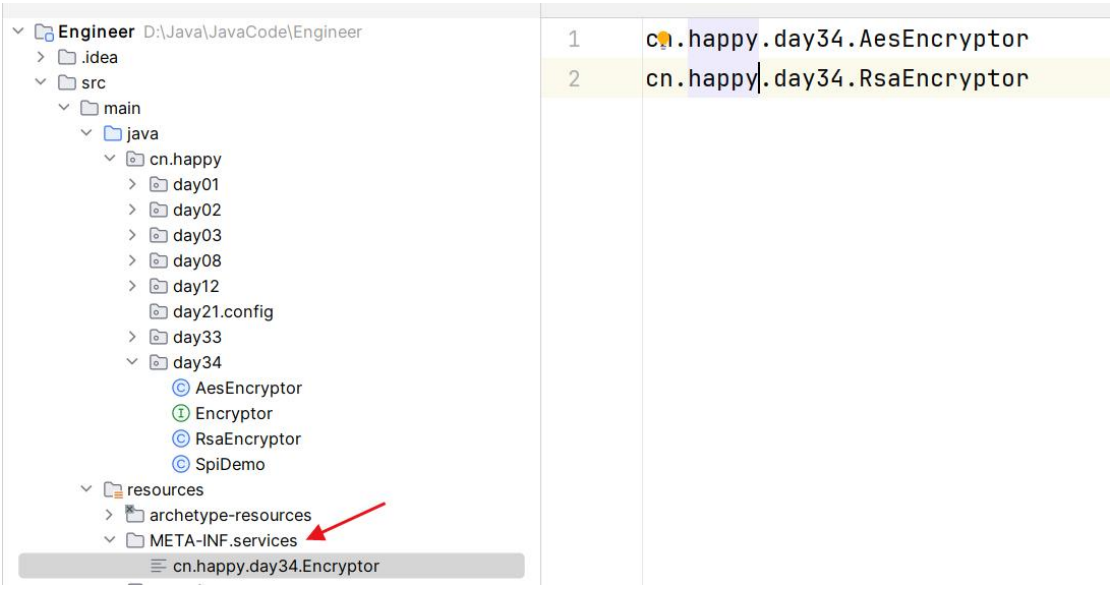
文件内容：

```
com.example.AesEncryptor
com.example.RsaEncryptor
```

4. 通过 ServiceLoader 加载服务

```java
public class SpiDemo {
    public static void main(String[] args) {
        ServiceLoader<Encryptor> encryptors = ServiceLoader.load(Encryptor.class);
        for (Encryptor encryptor : encryptors) {
            System.out.println(encryptor.encrypt("Hello SPI!"));
        }
    }
}

// 输出结果：
// AES_ENCRYPTED:Hello SPI!
// RSA_ENCRYPTED:Hello SPI!
```



五、SPI 的典型应用场景

1. JDBC 驱动加载

传统方式：需手动注册驱动类（如 Class.forName("com.mysql.jdbc.Driver")）。

SPI 方式：JDK 6+ 后，驱动 JAR 包通过 META-INF/services/java.sql.Driver 自动注册。

2. 日志框架桥接（如 SLF4J）

SLF4J 通过 SPI 发现具体的日志实现（Logback、Log4j2 等）。

配置文件示例：META-INF/services/org.slf4j.impl.StaticLoggerBinder

3. Spring Boot 自动配置

Spring Boot 的 spring.factories 文件本质是 SPI 的扩展。

文件路径：META-INF/spring.factories

内容示例：

```
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
com.example.MyAutoConfiguration
```

六、SPI 的优缺点分析

优点	缺点
解耦接口与实现，支持热插拔	所有实现类会被一次性加载，可能浪费资源
无需修改代码即可扩展功能	无法按需加载特定实现
天然支持多实现（如不同厂商驱动）	缺乏依赖管理机制（如版本冲突）

七、总结

SPI 的核心价值在于实现面向扩展的开闭原则（对扩展开放，对修改关闭）。其典型应用场景包括：

需要支持多厂商实现的框架（如数据库驱动、支付网关）。

希望用户能自定义扩展功能的系统（如 IDE 插件机制）。

模块化架构中动态加载组件。

面试篇之场景题

35、Java 中的对象一定在堆上分配吗

【腾讯】

在 Java 中，对象的内存分配通常发生在堆上，但并非绝对。

1. 常规情况：堆分配

堆是对象分配的主要区域：Java 通过 new 关键字创建的对象实例和数组，默认在堆内存中分配。堆由垃圾回收器（GC）管理，生命周期由 JVM 的垃圾回收策略决定。

线程共享与同步开销：堆是线程共享的，对象分配可能涉及同步机制（如 TLAB 优化），但本质上仍在堆内。

2. 优化场景：栈分配与标量替换

逃逸分析（Escape Analysis）：

JVM（如 HotSpot）在 JIT 编译时，会分析对象的作用域。若对象未逃逸（即仅在方法内部使用，未被外部引用或线程共享），可能进行优化。

栈上分配：对象直接在栈帧中分配，随方法结束自动销毁，无需 GC 介入。

标量替换（Scalar Replacement）：将对象拆解为基本类型（标量），存储在栈帧或寄存

器中，避免对象整体分配。

```
public void example() {
    Point p = new Point(1, 2); // 未逃逸对象
    System.out.println(p.x + p.y);
}
```

此处 Point 对象可能被分解为两个局部变量 x 和 y，无需在堆中分配。

3. 技术限制与实际情况

优化条件：逃逸分析需 JVM 支持且对象确实未逃逸。HotSpot 默认启用该优化，但复杂对象或特定代码结构可能导致优化失败。

性能权衡：逃逸分析本身消耗资源，JVM 会根据代码特征决定是否应用优化。小对象、高频调用方法更易受益。

4. 其他相关机制

TLAB（Thread-Local Allocation Buffer）：堆内为每个线程划分的私有区域，用于快速分配对象，减少锁竞争，但仍属于堆内存。

大对象直接进入老年代：某些 JVM 对超大对象（如长数组）直接分配在老年代，避免频繁 GC，但仍在堆内。

总结

Java 对象通常分配在堆上，但在 JVM 通过逃逸分析优化后，未逃逸对象可能栈分配或标量替换，从而避免堆内存开销。开发者通常无需关注此细节，优化由 JVM 自动完成。因此，严格来说，Java 对象不绝对在堆上分配，但大多数情况下确实如此。

面试篇之场景题

36、说一下你对行锁、临键锁、间隙锁的理解

【京东】

在 MySQL 的 InnoDB 引擎中，行锁、间隙锁和临键锁是处理并发事务和保证数据一致性的核心机制。

1. 行锁（Record Lock）

定义：行锁直接锁定索引中的某一条具体记录（即使表没有索引，InnoDB 也会隐式创建聚簇索引）。

作用场景：

对单行记录的增删改操作（如 UPDATE table SET ... WHERE id=1）。

显式加锁（如 SELECT ... FOR UPDATE 或 SELECT ... LOCK IN SHARE MODE）。

特点：

锁定已存在的记录：仅针对数据存在的行，不涉及间隙。

细粒度锁：仅阻塞对同一行的并发写操作，不影响其他行的操作。

依赖索引：若未使用索引，可能退化为表锁。

```
-- 事务A
BEGIN;
UPDATE users SET name='Alice' WHERE id=5; -- 对id=5的行加行锁
-- 事务B尝试修改同一行会被阻塞
UPDATE users SET name='Bob' WHERE id=5;
```

2. 间隙锁（Gap Lock）

定义：间隙锁锁定索引记录之间的间隙（即两个索引值之间的范围），防止其他事务插入数据。

作用场景：

范围查询（如 SELECT * FROM table WHERE id BETWEEN 5 AND 10 FOR UPDATE）。

防止幻读，确保同一事务多次查询结果一致。

特点：

仅作用于可重复读（RR）隔离级别：读已提交（RC）隔离级别下不生效。

允许多个间隙锁共存：不同事务可以锁定不同间隙。

```
-- 事务A
BEGIN;
SELECT * FROM users WHERE id BETWEEN 5 AND 10 FOR UPDATE; -- 锁定(5,10)的间隙
-- 事务B尝试插入id=8的记录会被阻塞
INSERT INTO users (id, name) VALUES (8, 'Charlie');
```

3. 临键锁（Next-Key Lock）

定义：临键锁是行锁与间隙锁的组合，锁定索引记录及其之前的间隙（左开右闭区间）。

作用场景：

默认的 InnoDB 锁机制，用于范围查询和防止幻读。

特点：

左开右闭区间：例如，索引值为 10 的记录，临键锁的锁定范围是前一个索引值到当前索引值的区间 (5, 10]（假设前一个索引是 5）。

覆盖范围：

锁定当前记录本身（行锁）。

锁定当前记录与前一个记录之间的间隙（间隙锁）

平衡并发与一致性：既防止其他事务修改当前记录，也阻止在间隙中插入新数据。

现有记录：3, 7, 10, 15

对 id=10 加临键锁时：

锁定区间：(7, 10]（即 7 < id ≤ 10）。

效果：

其他事务无法插入 id=8 或 id=9（间隙锁）。

其他事务无法修改或删除 id=10（行锁）。

退化为行锁或间隙锁：在唯一索引的等值查询中，若记录存在，退化为行锁；若不存在，退化为间隙锁。

```
-- 事务A
BEGIN;
SELECT * FROM users WHERE id > 5 AND id < 15 FOR UPDATE; -- 锁定(5,15)的间隙及记录
-- 事务B插入id=12或修改id=10均会被阻塞
```

面试篇之场景题

37、生产环境服务器变慢，如何诊断处理?

【阿里】

在生产环境中，服务器变慢是一个复杂的综合性问题，需要从系统资源、应用性能、网络状态、日志分析等多个维度进行排查。

一、快速定位瓶颈方向

1. 全局资源监控

```
# 实时查看系统资源
top
# 查看磁盘I/O统计
iostat -x 1
# 查看内存和swap使用
free -h
# 网络连接统计
sar -n DEV 1
```

2. 初步判断瓶颈类型

CPU 瓶颈：%us 【用户空间程序的 cpu 使用率】高（应用代码问题）、%sy【系统空间 CPU 利用率】高（系统调用频繁）、%wa【CPU 等待 I/O 操作完成的时间百分比】高（磁盘 I/O 阻塞）。

内存瓶颈：free 内存不足、swap 频繁交换（OOM 风险）。

磁盘瓶颈：%util 【利用率】> 80%、await 值高。

网络瓶颈：带宽占满、TCP 重传率高。

二、逐层深入排查

1. 进程级分析

高 CPU 进程：

```
# 按CPU排序进程
top -c -o %CPU
# 查看进程的线程CPU使用
pidstat -t -p <PID> 1
# 生成Java进程火焰图（需安装perf/async-profiler）
./profiler.sh -d 30 -f flamegraph.html <PID>
```

高内存进程：

```
# 按内存排序进程
top -c -o %MEM
# 查看进程内存映射
pmap -x <PID>
# Java应用内存分析（jmap + MAT）
jmap -dump:format=b,file=heap.hprof <PID>
```

2. I/O 瓶颈分析

磁盘 I/O：

```
# 查看磁盘I/O负载
iotop
# 定位高I/O的进程
pidstat -d 1
# 检查文件系统是否损坏
fsck /dev/sda1
```

网络 I/O：

```
# 查看网络连接状态
ss -s
# 检查带宽占用（iftop/nload）
iftop -i eth0
# 抓包分析（tcpdump/Wireshark）
tcpdump -i eth0 -w capture.pcap
```

3. 应用级诊断

数据库性能：

```
# 慢查询日志（MySQL）
SHOW FULL PROCESSLIST;
EXPLAIN SELECT ...;
# 锁等待分析
SHOW ENGINE INNODB STATUS;
```

JVM 性能：

```
# GC日志分析
jstat -gcutil <PID> 1000
# 线程转储（死锁检测）
jstack <PID> > thread_dump.txt
```

Web 服务：

```
# Nginx/Apache请求统计
tail -f /var/log/nginx/access.log | awk '{print $7}' | sort | uniq -c | sort -nr
# 接口响应时间（APM工具：SkyWalking、Arthas）
```

这样，你就可以实时看到哪个 IP 地址访问 Nginx 的次数最多。

三、日志与事件分析

1. 系统日志

关键日志路径：

/var/log/messages（系统事件）

/var/log/syslog

dmesg（内核日志）

排查方向：

硬件错误（磁盘坏道、内存故障）。

OOM Killer 日志（grep "Out of memory" /var/log/messages）。

内核崩溃（kernel panic）。

2. 应用日志

常见日志：

错误堆栈（如 Java 的 NullPointerException）。

慢请求（如 HTTP 500、数据库超时）。

```
# 实时监控日志
tail -f /path/to/app.log
# 日志聚合分析（ELK/Splunk）
grep "ERROR" app.log | awk '{print $4}' | sort | uniq -c
```

四、配置与安全排查

1. 系统配置

资源限制：

```
# 检查文件句柄数
ulimit -n
# 检查进程数限制
cat /proc/sys/kernel/pid_max
```

内核参数：

```
# 调整TCP连接回收（避免TIME_WAIT过多）
sysctl -w net.ipv4.tcp_tw_reuse=1
# 调整文件描述符限制
sysctl -w fs.file-max=65536
```

2. 安全审查

入侵检测：

```
# 检查异常进程
ps aux | grep -E "(挖矿程序名)"
# 检查定时任务
crontab -l
# 检查SSH登录记录
lastb | head -n 20
```

资源滥用：

排查是否被恶意爬虫或 DDoS 攻击（这种攻击形式主要通过恶意流量消耗网络或网络设备

的资源）。

总结

监控全局资源：快速定位 CPU、内存、磁盘、网络中的瓶颈点。

深入分析进程：找到具体占用资源的进程或线程。

检查日志与配置：排查系统错误、应用异常及配置不合理项。

结合应用特性：数据库、缓存、代码逻辑针对性优化。

关键思维：

先整体后局部：先看全局资源，再聚焦到具体进程或代码。

工具链熟悉度：掌握 Linux 命令、APM（应用程序性能管理）工具、日志分析能力。

数据驱动决策：通过指标和日志定位问题，而非盲目猜测。

面试篇之场景题

38、HashMap 啥时候扩容，为什么扩容?

【京东】

HashMap 的扩容机制是其高效性能的核心设计之一。

一、触发扩容的条件

1. 元素数量超过阈值

阈值计算公式：阈值 = 当前容量（capacity） × 负载因子（loadFactor，默认 0.75）。

例如，默认初始容量为 16，阈值则为 16 × 0.75 = 12。当元素数量（size）超过 12 时，触发扩容。

在 putVal() 方法中，每次插入元素后检查 size 是否超过阈值。

2. 链表转红黑树时容量不足（JDK 1.8+）

条件：当链表长度 ≥ 8（TREEIFY_THRESHOLD）且数组长度 < 64（MIN_TREEIFY_CAPACITY）时，优先扩容而非树化。

目的：通过扩容分散冲突，避免链表过长导致的查询性能下降。

二、扩容的核心流程

1. 计算新容量

规则：新容量为旧容量的 2 倍（newCap = oldCap << 1），但不超过最大容量（MAXIMUM_CAPACITY = 1<<30）。

```
if (oldCap > 0) {
    if (oldCap >= MAXIMUM_CAPACITY) {
        threshold = Integer.MAX_VALUE;
        return oldTab;
    }
    else if ((newCap = oldCap << 1) < MAXIMUM_CAPACITY)
        newThr = oldThr << 1; // 阈值也翻倍
}
```

2. 创建新数组

使用新容量创建新数组 newTab：

```
Node<K,V>[] newTab = (Node<K,V>[])new Node[newCap];
```

3. 数据迁移

链表迁移优化（JDK 1.8+）：

高低位拆分：通过 (e.hash & oldCap) == 0 判断元素是否留在原位置（低位链表）或迁移到新位置（高位链表，原索引 + oldCap）。

```
if ((e.hash & oldCap) == 0) {
    // 加入低位链表
} else {
    // 加入高位链表
}
```

优势：避免重新计算哈希，时间复杂度从 $O(n^2)$ 降为 $O(n)$。

红黑树迁移：若树节点数 ≤ 6，退化为链表。

三、为什么需要扩容?

1. 避免哈希冲突过多

哈希冲突：当多个键的哈希值映射到同一桶时，链表或红黑树的查询效率会下降（链表为 O(n)，红黑树为 O(logn)）。

扩容作用：通过扩大数组容量，分散元素到更多桶中，减少冲突概率。

2. 维持性能平衡

负载因子的意义：默认值 0.75 是空间与时间的折中。

值过小（如 0.5）：空间浪费，扩容频繁。

值过大（如 0.9）：冲突增多，查询效率降低。

3. 数据一致性保证

扩容在插入前完成：在 putVal() 方法中，先调用 resize() 扩容，再插入元素。

```
if ((tab = table) == null || (n = tab.length) == 0)
    n = (tab = resize()).length; // 初始化或扩容
```

原因：若插入后再扩容，元素可能被分配到错误的桶中，导致数据不一致。

面试篇之场景题

39、如何实现登录用户可以存 1000 sku,未登录用户只能存 200 sku

【京东】

一、用户身份判断

1、前端身份验证

用户访问时，前端通过以下方式判断登录状态：

检查浏览器本地存储（LocalStorage/SessionStorage）中是否存在登录凭证（如 Token），

也可以检查 Cookie 中是否存在服务端会话 ID。

若存在有效凭证，标记为已登录用户；否则视为未登录用户（游客）

2、后端身份验证

所有涉及数据存储的 API 请求，后端需强制校验身份：

登录用户：验证 Token 或 Session 有效性

二、存储方案设计

1. 未登录用户（游客）

前端存储

使用浏览器 LocalStorage 存储 SKU 数据（上限 200 条）：

创建专用存储键（如 guest_skus）

每次新增 SKU 时：

（1）读取现有数据并转为数组

（2）检查数组长度是否≥200

（3）若超限，阻止存储并提示登录

（4）数据格式建议加密存储（如 AES 加密）

2. 登录用户

服务端存储

1、数据库设计专用 SKU 表，包含字段：

user_id（关联用户）

sku_id（商品标识）

created_at（创建时间）

2、每次新增 SKU 时：

查询当前用户的 SKU 总数（SELECT COUNT(*) WHERE user_id=xxx）

若总数≥1000，返回错误提示

未超限时执行插入操作

缓存优化：使用 Redis 缓存用户 SKU 计数，避免频繁查询数据库

三、数据迁移流程（登录时）

1、触发时机

用户完成登录后，前端自动触发迁移流程

2、执行步骤

步骤 1：读取本地游客数据

从 LocalStorage 获取 guest_skus 内容

步骤 2：计算可迁移空间

调用服务端接口获取用户当前 SKU 数量（如 850 条）

剩余空间 = 1000 - 850 = 150 条

步骤 3：筛选迁移数据

截取前 150 条游客 SKU 数据

剩余 50 条保留在本地（或提示用户手动处理）

步骤 4：服务端批量写入

调用批量插入 API 完成迁移

步骤 5：清理本地数据

删除已成功迁移的 SKU 记录

3、异常处理

（1）迁移失败时保留本地数据供重试

（2）记录迁移日志供后续排查

四、技术验证流程

关键测试用例

（1）未登录用户连续添加 200 个 SKU 后无法继续新增

（2）登录用户添加 1000 个 SKU 后返回错误

（3）登录后游客数据自动迁移且总数不超过 1000

（4）清除浏览器数据后游客 SKU 重置

性能压测

（1）模拟 1000 并发用户同时进行 SKU 操作

（2）验证 Redis 计数缓存对数据库查询的优化效果

面试篇之场景题

40、创建索引的时候需要考虑哪些问题?

【腾讯】

创建索引是数据库性能优化的核心手段之一，但索引并非越多越好。需要从多个维度综合考量，做出平衡和最优决策。

一、 数据维度（理解你的数据）

1、表的大小：

小表（几千行）：索引收益可能不大，甚至全表扫描更快，维护索引反而增加开销。

大表（百万/千万+行）：索引对查询性能提升至关重要。

2、数据分布与基数：

基数： 列中不同值的数量。

高基数： 列值唯一或接近唯一（如主键、用户名、邮箱）。非常适合创建索引，过滤效率高。

低基数： 列值重复度高（如性别、状态标志）。索引效果差，可能不如全表扫描。有时特殊场景（如数据仓库的位图索引）除外。

数据倾斜： 某些值出现频率极高。即使基数高，查询如果总命中最频繁的值，索引可能失效（优化器可能选择全扫）。

3、数据更新频率：

频繁插入/更新/删除的表：每个写操作都需要更新相关的索引，会带来显著的写性能开销。

需要谨慎评估索引数量和复杂度。

读多写少的表：索引带来的读性能提升通常远大于写开销，可以适当多建索引。

4、NULL 值比例：

高比例的 NULL 值可能影响索引效率（取决于数据库实现和索引类型）。考虑索引是否包含 NULL 值或使用 IS NULL 条件。

二、查询需求维度

1、高频查询条件

常出现在 WHERE、JOIN ON、ORDER BY/GROUP BY 中的列优先建索引。

2、查询类型匹配

等值查询（=）：适合 B+树、哈希索引。

范围查询（>, <, BETWEEN）：需 B+树索引。

模糊查询（LIKE 'prefix%'）：可用 B+树索引；LIKE '%suffix'需全文索引。

3、覆盖索引优化

索引包含查询所需全部字段时，避免回表，性能最优。

三、索引设计维度

单列索引 vs 组合索引

组合索引：A，B，C

遵循最左前缀原则（索引 (a,b,c) 可优化 a=?、a=? AND b=?，但无法优化 b=?）。

列顺序优先级：高基数列 > 等值查询列 > 范围查询列。

单列索引：易维护，但多条件查询时可能触发索引合并（效率较低）。

四、性能与成本维度

1、写性能代价

INSERT/UPDATE/DELETE 需同步更新索引，索引越多，写操作越慢。高频写入场景需严格控制索引数量。

2、存储空间开销

索引可能占用与原始表相当的磁盘空间（尤其是组合索引）。

3、索引维护成本

数据变更导致索引碎片化，需定期 REBUILD INDEX

五、最佳实践原则

1、少而精策略

优先为 高频+高收益查询 创建索引，避免冗余索引。

2、持续监控优化

定期检查索引使用率（如 sys.dm_db_index_usage_stats），删除无用索引。

3、测试驱动决策

通过 EXPLAIN 分析执行计划，验证索引有效性。

4、业务场景适配

OLTP 系统：侧重读写平衡，索引不宜过多。

OLAP 系统：侧重查询速度，可适当增加索引。

面试篇之场景题

41、你平时是怎么查看日志的?

【腾讯】

在 Java 项目中，我主要通过以下方式高效查看和分析日志，按照操作场景分类说明：

一、实时日志监控（动态跟踪）

1、基础命令

```
tail -f app.log                    # 实时追踪最新日志
tail -f app.log | grep "ERROR"     # 实时过滤错误日志
```

2、多文件追踪

multitail app.log -I api.log　　# 同时监控多个日志文件

api.log 的内容会被添加到已经打开的窗口中，而不是创建一个新的窗口

tail -f service_*.log　　# 通配符匹配多个服务日志

二、历史日志检索（静态分析）

1、关键词搜索

grep "NullPointer" app.log　　# 基础搜索

grep -C 10 "OrderID:123" app.log　　# 显示匹配行前后 10 行（Context）

grep -r "ConnectionTimeout" /var/log/　　# 递归搜索目录

2、多条件过滤

grep -E "ERROR|WARN" app.log　　# 同时搜索 ERROR 或 WARN

grep "PaymentService" app.log | grep "FAILED"　　# 管道组合过滤

三、高级日志分析（复杂场景）

1、时间范围筛选

提取 9:00-10:00 的日志

sed -n '/2025-05-29 09:00:00/,/2025-05-29 10:00:00/p' app.log

2、统计分析与聚合

awk -F'\t' '/ERROR/{print $6}' app.log | sort | uniq -c | sort -nr　　# 错误类型 TOP10

cat app.log | awk '{print $8}' | sort -n | tail -10　　# 耗时最长的 10 个请求

3、JSON 日志处理

cat app.log | jq '. | select(.level=="ERROR") | .message'　　# 解析结构化日志

仅提取级别为 "ERROR" 的日志消息，并且包含一个字段 level 和一个字段 message。

四、可视化工具（大型系统）

工具	使用场景	关键操作
ELK Stack	分布式系统日志聚合	时间直方图分析
Grafana Loki	云原生环境日志监控	LogQL 查询： `{app="order"}= "Timeout"
阿里云 SLS	云服务日志分析	SELECT status, COUNT(1) GROUP BY status

五、链路追踪（微服务场景）

1、TraceID 定位

```
grep "trace_id=0x7a3b" *.log     # 跨服务追踪全链路
```

用于在当前目录下所有.log 后缀的文件中搜索包含"trace_id=0x7a3b"字符串的行

2、时序重组

```
grep "order_123" order.log payment.log inventory.log | sort -k 4   # 按时间戳排序
```

用于追踪一个特定订单(order_123)在系统中的完整生命周期，包括订单创建、支付处理和库存变动

六、效率技巧

1、大文件处理

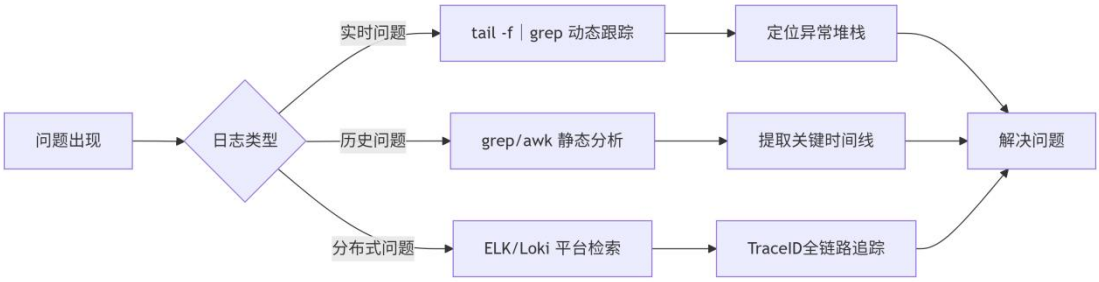
```
less +G app.log                # 直接跳转文件末尾（GB 级文件秒开）
zgrep "OOM" *.log.gz         # 直接搜索压缩包内容
```

用于在当前目录下所有.gz 压缩的日志文件中搜索包含"OOM"(Out Of Memory)字符串的行

2、色彩高亮

tail -f app.log | ccze -A # 实时日志着色输出

这个命令组合用于实时监控日志文件并彩色化输出



面试篇之场景题

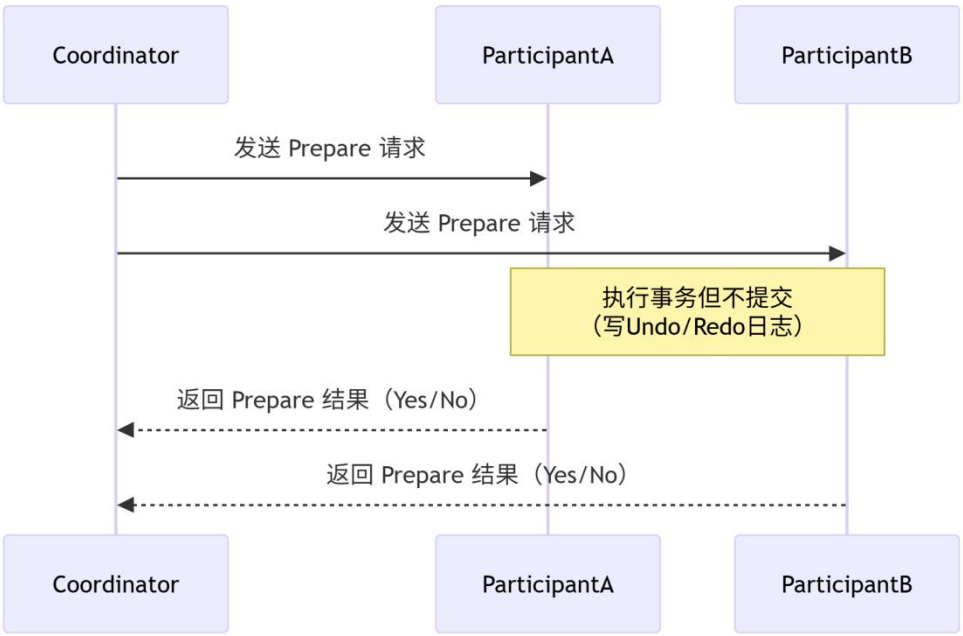
42、什么是二阶段提交?

【百度】

二阶段提交（Two-Phase Commit, 2PC）是一种分布式事务协议，用于在分布式系统中确保多个参与者（如数据库、服务）的事务原子性。其核心思想是通过一个协调者（Coordinator）分两个阶段协调所有参与者（Participants）的操作，最终实现所有节点要么全部提交事务，要么全部回滚事务。

以下用序列图形式描述 2PC 流程，假设场景：协调者（Coordinator）管理两个参与者（Participant A 和 Participant B）。

第一阶段：准备阶段（Prepare Phase）



步骤说明：

协调者向所有参与者发送 Prepare 请求。

参与者执行事务操作（更新数据），但不提交，将操作写入 Undo/Redo 日志（用于故障恢复）。

参与者根据自身状态回复：

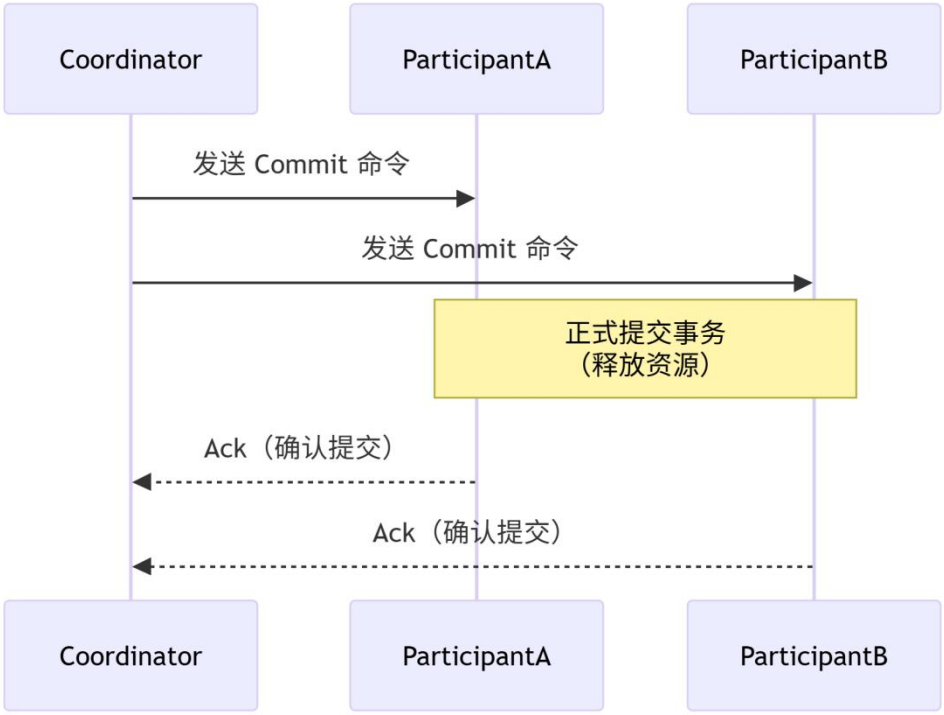
Yes：事务可提交（资源锁定，进入阻塞状态）。

No：事务需中止（如违反约束）。

第二阶段：提交阶段（Commit Phase）

根据第一阶段的结果，分两种情况：

情况 1：全部参与者返回 Yes → 提交事务



步骤说明：

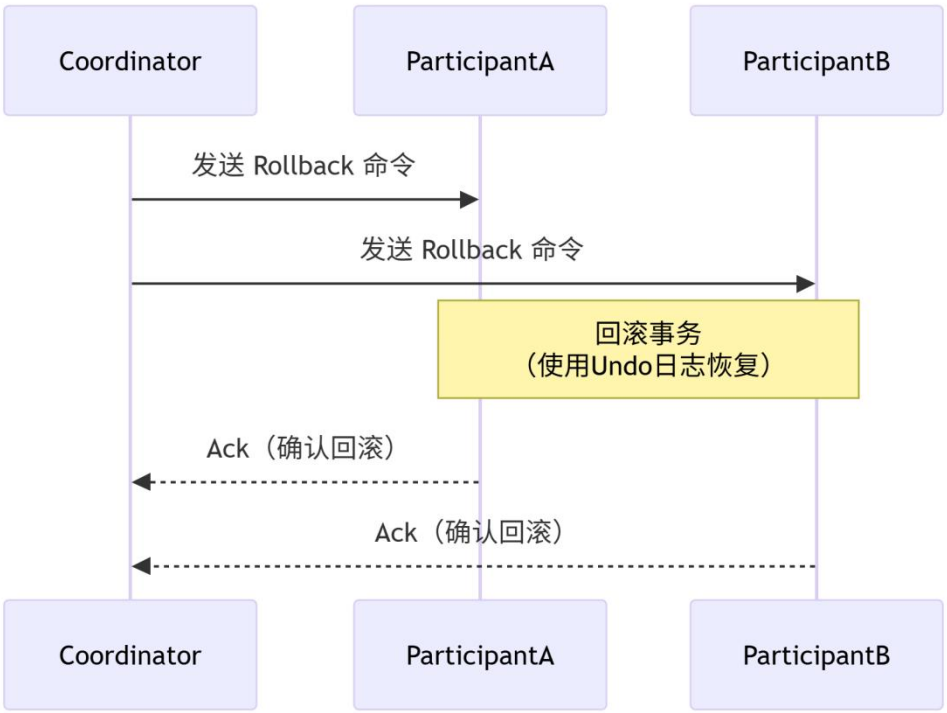
协调者向所有参与者发送 Commit 命令。

参与者正式提交事务，释放资源锁。

参与者发送 Ack 确认。

协调者收到所有 Ack 后完成事务。

情况 2：任意参与者返回 No 或超时 → 回滚事务



步骤说明：

协调者向所有参与者发送 Rollback 命令。

参与者使用 Undo 日志回滚事务，释放资源锁。

参与者发送 Ack 确认。

协调者收到所有 Ack 后终止事务。

关键特点

原子性保证：

所有参与者要么全部提交，要么全部回滚。

同步阻塞：

参与者在第一阶段后进入阻塞状态（锁定资源），直到收到协调者命令。

单点故障风险：

若协调者故障，参与者可能永久阻塞。

若参与者故障，协调者需等待其恢复（通过日志恢复状态）。

数据不一致场景：

协调者发送 Commit 后部分参与者崩溃 → 部分提交、部分未提交（需人工介入）。

二阶段提交通过预提交+正式提交的分阶段设计，为分布式事务提供强一致性保证，但存在阻塞和单点故障问题。在实际开发中，需结合业务场景权衡一致性与可用性，或选择优化方案（如三阶段提交、TCC 柔性事务）。

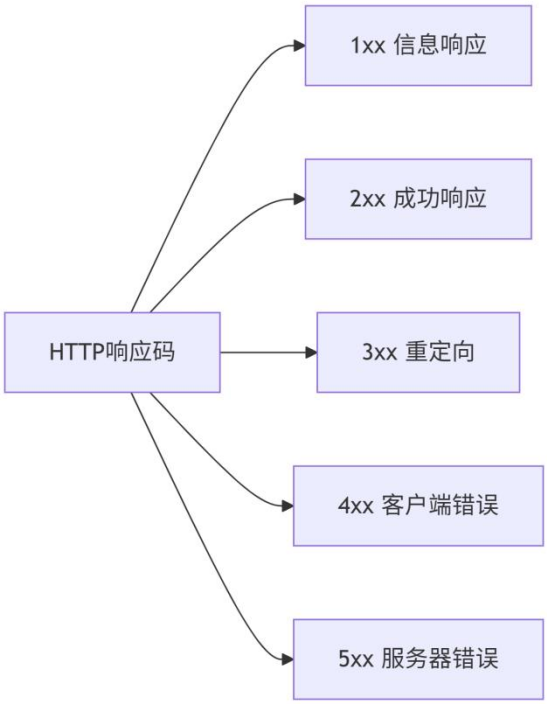
面试篇之场景题

43、HTTP 响应码有哪些？分别代表什

么含义?

【百度】

HTTP 响应码（状态码）是服务器对客户端请求的反馈结果，用于表示请求的处理状态。根据第一位数字分为五大类，每类有特定的语义和常见子状态码。



一、1xx 信息响应（临时状态）

服务器已收到请求，需客户端继续操作：

100 Continue：客户端应继续发送请求剩余部分（如大文件上传）

101 Switching Protocols：服务器同意切换协议（如从 HTTP 升级到 WebSocket）

二、2xx 成功响应（请求已处理）

请求被成功接收并处理：

200 OK：标准成功状态（GET/POST 请求成功）

201 Created：资源创建成功（如 POST 新建用户后返回新资源 URL）

204 No Content：无返回内容（如 DELETE 请求成功）

206 Partial Content：部分内容响应（支持断点续传）

三、3xx 重定向（需进一步操作）

资源位置变化，需客户端重定向：

301 Moved Permanently：资源永久迁移（SEO 权重转移）

302 Found：资源临时迁移（浏览器自动跳转新 URL）

304 Not Modified：资源未修改（客户端可使用缓存）

307 Temporary Redirect：临时重定向且保持原请求方法（如 POST 重定向后仍用 POST）

四、4xx 客户端错误（请求问题）

因客户端请求错误导致失败：

400 Bad Request：请求语法错误（如参数格式无效）

401 Unauthorized：未认证（需提供 Token 或密码）

403 Forbidden：无权限访问（认证成功但授权不足）

404 Not Found：资源不存在（URL 错误或资源已删除）

405 Method Not Allowed：HTTP 方法不被支持（如对只读资源用 POST）

五、5xx 服务器错误（服务器问题）

服务器处理请求时内部异常：

500 Internal Server Error：通用服务器错误（如代码崩溃）

502 Bad Gateway：网关/代理从上游收到无效响应（如反向代理后端服务故障）

503 Service Unavailable：服务不可用（过载或维护中）

504 Gateway Timeout：网关等待上游响应超时

六、高频场景总结

场景	推荐状态码	示例
成功获取数据	200 OK	查询用户信息成功
身份验证失败	401 Unauthorized	Token 过期或未登录
权限不足	403 Forbidden	普通用户访问管理员接口
服务器过载	503 Service Unavailable	流量激增时返回“稍后重试

面试篇之场景题

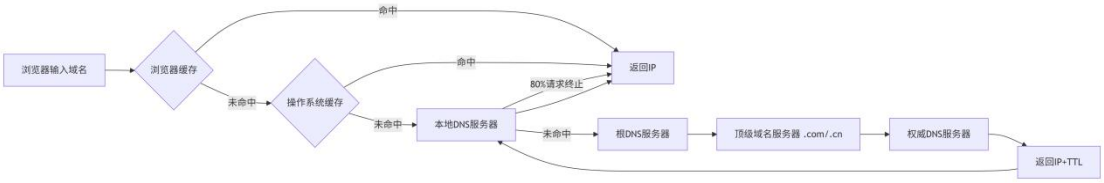
44、说说域名解析详细过程?

【百度】

域名解析（DNS 解析）是将人类可读的域名（如 www.example.com）转换为机器可识别的 IP 地址 (如 192.168.1.1) 的过程。其核心依赖分布式数据库系统 DNS (Domain Name System），通过多层服务器协作完成。

一、域名解析的核心步骤

以下流程模拟分层递进的“解析链”，前两步在用户设备本地完成，后七步依赖外部 DNS 服务器：



1. 本地阶段：快速缓存查询

（1）浏览器缓存检查

浏览器优先检查自身缓存（如 Chrome 的 net-internals）。若域名近期访问过，直接返回 IP，解析终止。缓存时间受 TTL（Time to Live）控制，通常几分钟至几小时。

（2）操作系统缓存与 Hosts 文件

若浏览器无缓存，查询操作系统缓存（如 Windows 的 DNS Client 服务）。

检查 Hosts 文件（路径：

Windows: C:\Windows\System32\drivers\etc\hosts

Linux: /etc/hosts

用户可在此手动绑定域名与 IP（常用于测试或屏蔽恶意网站）

2. 服务器阶段：分布式 DNS 查询

（3）本地 DNS 服务器请求

若本地无缓存，请求发送至 本地 DNS 服务器（通常由 ISP 提供，如 223.5.5.5）。

关键作用：承担 80% 的解析请求（因缓存常见域名）

（4）根 DNS 服务器查询

本地 DNS 未命中时，向 根 DNS 服务器（全球 13 组，按 A-M 编号）发送请求。根服务器返回顶级域（如 .com）的服务器 IP

（5）顶级域名服务器（TLD）查询

本地 DNS 根据后缀（如 .com）向 TLD 服务器请求，TLD 返回负责该域名的权威 DNS 服务器地址

（6）权威 DNS 服务器查询

权威服务器 (如域名注册商提供的 NS 服务器) 存储域名的精确解析记录，返回 IP 及 TTL 值

（7）结果缓存与返回

本地 DNS 服务器缓存 IP（按 TTL 过期），并返回结果给操作系统。

操作系统缓存结果后，传递至浏览器

3. 最终访问

浏览器获得 IP，发起 HTTP 请求访问目标服务器，完成页面加载

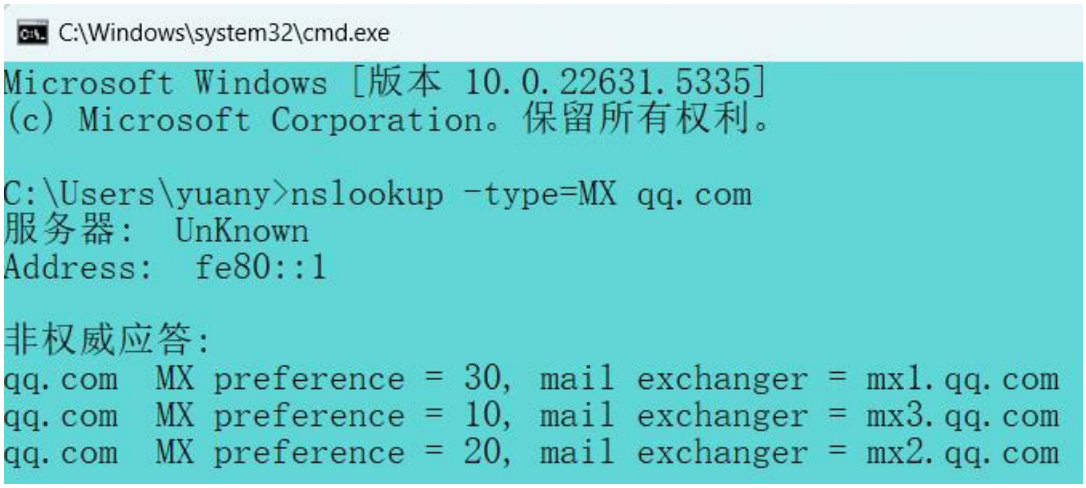
二、关键解析记录类型

权威 DNS 服务器存储多种记录，常见类型包括：

记录类型	作用	示例
A 记录	域名 → IPv4 地址	example.com → 192.0.2.1
AAAA 记录	域名 → IPv6 地址	example.com → 2001:db8::1

CNAME 记录	域名别名 → 目标域名	www.example.com→ example.com
MX 记录	邮件服务器地址指向	qq.com → mx1.qq.com
NS 记录	指定域名的权威 DNS 服务器	example.com → ns1.cloud.com

示例：查询腾讯邮箱解析



如果需要将子域名交给其他 DNS 服务商解析，则需要添加 NS 记录。

NS 记录的记录值，请填写要授权 DNS 服务器的域名。

域名解析是互联网访问的“无声桥梁”，其核心是 分层查询 与 分布式缓存 机制：

1、本地缓存（浏览器/OS）→ 2. 本地 DNS（ISP 服务器）→ 3. 根/TLD 服务器 → 4. 权威 DNS → 5. 返回 IP。

面试篇之场景题

45、了解过适配器模式么？

【阿里】

适配器模式是一种结构型设计模式，用于解决接口不兼容问题，使原本无法协同工作的类能够协同工作。其核心思想是通过一个中间层（适配器）转换接口，类似于现实世界的电源适配器。

一、模式结构（三个关键角色）

1、目标接口（Target）

客户端期望的接口规范

例如：USB 接口规范（包含 transferData() 方法）

2、被适配者（Adaptee）

已存在的、需要被复用的组件，但其接口与目标不兼容

例如：TypeCDevice 类（只有 transmitData() 方法）

3、适配器（Adapter）

实现目标接口，并持有被适配者的引用

在目标接口方法中调用被适配者的功能

例如：TypeCToUSBAdapter 实现 USB 接口，内部调用 TypeCDevice.transmitData()

（1）USB 接口

```
2 usages   1 implementation
public interface USB {
    1 usage   1 implementation
    public void transferData();
}
```

（2）被适配者

```
// 被适配者
3 usages
public class TypeCDevice {
    1 usage
    public void transmitData() {
        System.out.println("Type-C数据传输中...");
    }
}
```

（3）适配器

```java
// 适配器（对象适配器）
1 usage
public class TypeCToUSBAdapter implements USB {
    2 usages
    private TypeCDevice typeCDevice;

    1 usage
    public TypeCToUSBAdapter(TypeCDevice typeCDevice) {
        this.typeCDevice = typeCDevice;
    }

    1 usage
    @Override
    public void transferData() {
        typeCDevice.transmitData(); // 接口转换
    }
}
```

（4）测试

```java
public class T1 {
    public static void main(String[] args) {
        USB usb=new TypeCToUSBAdapter(new TypeCDevice());
        usb.transferData();
    }
}
```

二、实现方式对比

类型	类适配器	对象适配器
实现机制	继承被适配者 + 实现目标接口	组合被适配者 + 实现目标接口
灵活性	较低（Java 单继承限制）	较高（推荐）
代码耦合	紧耦合（直接继承）	松耦合（通过组合）

适配多个类	不支持（单继承限制）	支持（可组合多个 Adaptee）
示例代码	class Adapter extends Adaptee implements Target	class Adapter implements Target { private Adaptee adaptee; }

三、设计原理与本质

1、开闭原则（OCP）

在不修改原有代码（Adaptee）的情况下扩展功能

2、单一职责原则（SRP）

适配器只负责接口转换，业务逻辑仍由 Adaptee 处理

3、解耦思想

隔离客户端与底层实现，客户端只依赖 Target 接口

四、Java 标准库实例

// 字节流→字符流适配

Reader reader = new InputStreamReader(System.in);

InputStreamReader 是适配器（实现 Reader），内部封装 InputStream

面试篇之场景题

46、Maven 能为我们解决什么问题?

【腾讯】

Maven 不仅仅是一个构建工具，它更是一个项目管理和理解工具，旨在为 Java 项目提供一套标准化的构建生命周期、依赖管理、项目描述和最佳实践。

Maven 主要为我们解决以下关键问题：

1、依赖管理混乱

问题： 手动下载、管理、添加 JAR 包到类路径极其繁琐，版本冲突、依赖缺失、依赖传递性导致“JAR 地狱”。

解决： Maven 的核心功能是自动化依赖管理。

中央仓库： 提供海量开源库的统一存储和检索。

坐标体系： 通过 groupId, artifactId, version (GAV) 唯一标识依赖项。

依赖声明： 只需在 pom.xml 中声明所需依赖及其版本，Maven 自动从仓库下载（包括本地仓库缓存）并添加到项目的构建路径。

传递性依赖： 自动解析和下载依赖项所需的依赖项（传递依赖），极大地简化了复杂依赖

链的管理。

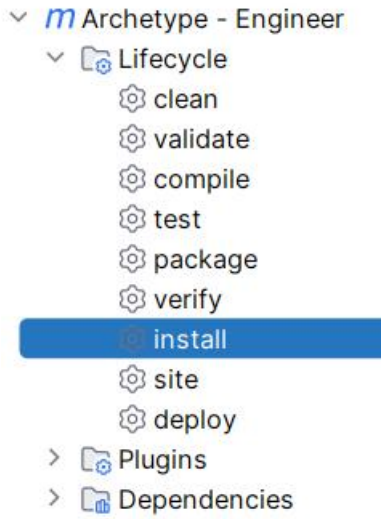
依赖范围： 通过 scope (compile, provided, test, runtime 等)精确控制依赖在哪些阶段（编译、测试、运行）生效，避免冗余和冲突。

2、构建过程不一致和复杂

问题：手动编写和维护 Ant 或其他脚本构建项目（编译、测试、打包、部署等）容易出错，脚本复杂且项目间不统一，新人上手成本高。

解决： Maven 引入了标准化的构建生命周期和插件机制。

预定义生命周期： 提供清晰、有序的构建阶段(validate, compile, test, package, install, deploy 等)。开发者只需执行一个目标(如 mvn package），Maven 会自动按顺序执行该目标之前的所有阶段。



约定优于配置： 定义了标准的项目目录结构（src/main/java, src/test/java, src/main/resources 等）。遵循约定，无需额外配置，Maven 就知道去哪里找源代码、资源文件、测试代码等。这大大简化了配置，并保证了项目结构的一致性。

插件化架构： 所有构建任务（编译、测试、打包 Jar/War、生成站点、发布等）都由插件

实现。Maven 核心只负责调度生命周期和插件执行。丰富的插件生态(maven-compiler-plugin, maven-surefire-plugin, maven-jar-plugin 等）覆盖了几乎所有构建需求。

简化构建命令： 复杂的构建过程简化为执行几个简单的、一致的生命周期命令或插件目标。

3、项目信息缺失和文档匮乏

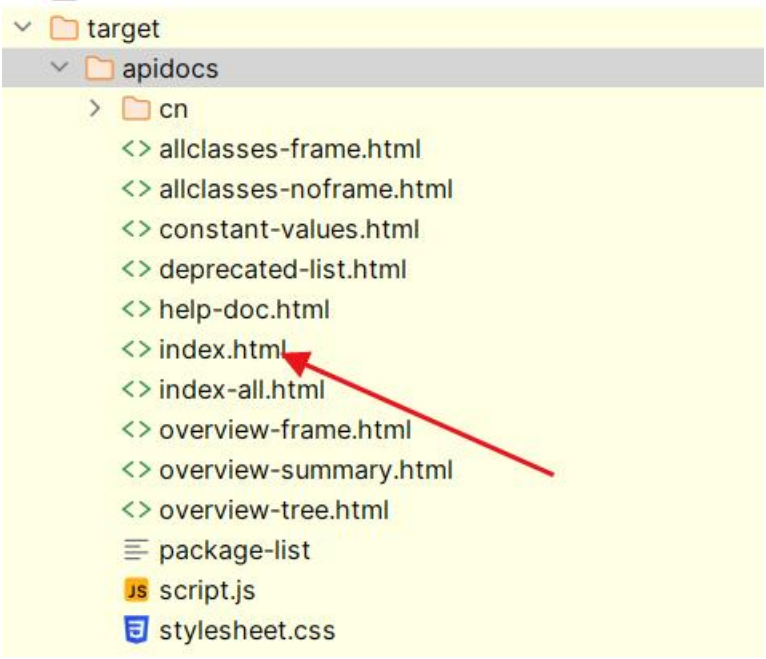
问题： 项目元数据（名称、描述、版本、开发者、许可证、SCM 信息等）分散或缺失，项目文档难以生成和维护。

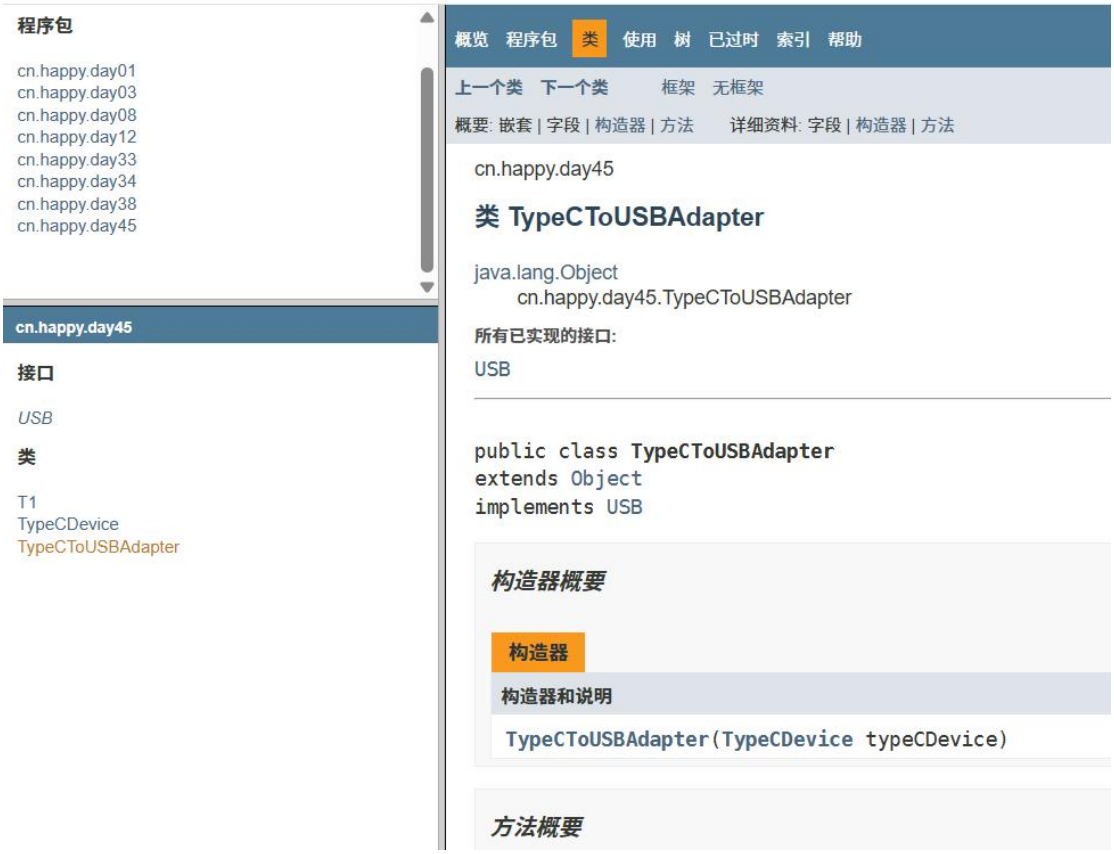
解决： pom.xml 作为单一的项目对象模型。

中心化元数据： pom.xml 文件是项目的核心描述文件，集中存储了项目标识(GAV)、依赖、构建配置、开发者信息、SCM 地址、许可证、项目描述等所有重要元数据。

项目报告： Maven 可以轻松生成项目网站，包含项目信息、依赖报告、测试报告、代码质量报告、Javadoc 等，极大改善了项目的透明度和可维护性。

```
[INFO] Building jar: D:\Java\JavaCode\Engineer\target\Engineer-1.0-SNAPSHOT-javadoc.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  18.109 s
[INFO] Finished at: 2025-06-04T13:06:49+08:00
[INFO] ------------------------------------------------------------------------
```





4、多模块项目管理困难

问题： 大型项目通常拆分为多个相互依赖的子模块，手动管理和构建它们非常复杂且容易出错。

解决： Maven 提供强大的多模块项目支持。

聚合： 定义一个父项目（通常打包类型为 pom），在父 pom.xml 的 <modules> 中列出所有子模块。

继承：子模块自动继承父 pom.xml 中的配置（如依赖管理 <dependencyManagement>、插件管理 <pluginManagement>、属性、仓库配置等），实现公共配置的复用和统一管理。

依赖管理简化： 父 POM 可以集中管理公共依赖的版本，子模块只需声明依赖的 groupId 和 artifactId，无需重复指定版本，确保版本一致性。

一键构建： 在父项目目录执行构建命令（如 mvn clean install），Maven 会根据模块依赖关系自动按顺序构建所有子模块。

5、促进一致性和最佳实践

问题： 团队或组织内项目结构、构建方式、依赖版本管理方式各异，导致维护成本高，知识共享困难。

解决： 通过约定、标准化和继承机制。

统一结构： 强制的标准目录结构使任何熟悉 Maven 的开发者都能快速理解项目。

统一构建流程： 标准生命周期确保所有项目都遵循相同的构建步骤。

公司级父 POM： 组织可以创建自己的“公司级父 POM”，定义组织范围内的统一依赖版本、仓库配置、插件配置、代码规范插件等。所有项目继承此父 POM，强制推行最佳实践和统一标准。

6、集成与扩展性

解决：

持续集成： 标准化的命令行接口和构建过程使其与 Jenkins, GitLab CI, TeamCity 等持续集成工具无缝集成。

IDE 支持： 所有主流 Java IDE (IntelliJ IDEA, Eclipse, NetBeans) 都对 Maven 项目提供开箱即用的深度支持（导入、依赖管理、运行构建等）。

丰富插件生态： 庞大的第三方插件库几乎可以满足任何特殊构建或集成需求（代码生成、Docker 打包、云部署等）。

总的来说，Maven 显著提升了 Java 项目的开发效率、构建可靠性、可维护性和团队协作能力，是现代 Java 生态系统中不可或缺的基础设施。

面试篇之场景题

47、如果一个线程两次调用 start()，会出现什么问题?

【京东】

在 Java 中，一个线程两次调用 start() 方法会抛出 IllegalThreadStateException 异常。

1. 根本原因：线程状态检查

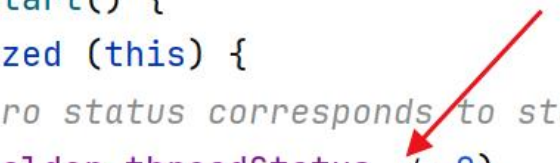
线程的生命周期包含 NEW (新建) 、RUNNABLE (可运行) 、BLOCKED (阻塞) 、WAITING

（等待）、TIMED_WAITING（超时等待） 和 TERMINATED（终止） 状态。

调用 start() 时，线程会从 NEW 状态进入 RUNNABLE 状态。

关键机制：start() 方法内部会检查线程的 threadStatus（状态标志）。如果状态不是 NEW（即已启动过），会直接抛出异常：

```
public void start() {
    synchronized (this) {
        // zero status corresponds to state "NEW".
        if (holder.threadStatus != 0)
            throw new IllegalThreadStateException();
        start0();
    }
}
```



2. 底层设计原理

线程唯一性：每个线程对象对应一个唯一的系统级线程资源（如操作系统线程句柄）。多次调用 start() 试图复用同一个线程资源，违反了线程生命周期设计。

资源管理：第一次调用 start() 时，JVM 会为线程分配栈空间、程序计数器等资源。第二次调用时，这些资源已被占用或已释放，无法重新初始化。

3. 异常示例

以下代码必现异常：

```
public class T1 {
    public static void main(String[] args) {
        Thread thread = new Thread(() -> System.out.println("Thread running"));
        thread.start(); // 第一次调用，正常执行
        thread.start(); // 第二次调用，抛出 IllegalThreadStateException
    }
}
```

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
Exception in thread "main" java.lang.IllegalThreadStateException Create breakpoint
    at java.base/java.lang.Thread.start(Thread.java:1525)
    at cn.happy.day47.T1.main(T1.java:7)
```

Java 通过抛出 IllegalThreadStateException 来强制线程的单次启动原则，这是为了保证线程状态的一致性和可预测性。正确的做法是每次执行新任务时创建新的线程实例或使用线程池。

面试篇之场景题

48、SimpleDateFormat 是线程安全的吗？为什么？

【京东】

SimpleDateFormat 不是线程安全的。

线程不安全的原因

1、内部可变状态：

SimpleDateFormat 内部维护了一个 Calendar 对象引用

```
public abstract class DateFormat extends Format {

    The Calendar instance used for calculating the date-time fields and the instant of time. This field
    is used for both formatting and parsing.
    Subclasses should initialize this field to a Calendar appropriate for the Locale associated with
    this DateFormat.
    protected Calendar calendar;
```

```
public SimpleDateFormat( @NonNls @NotNull String pattern,  @NotNull Locale locale)
 {
    if (pattern == null || locale == null) {
        throw new NullPointerException();
    }

    initializeCalendar(locale);
    this.pattern = pattern;
    this.formatData = DateFormatSymbols.getInstanceRef(locale);
    this.locale = locale;
    initialize(locale);
}
```

```
 private void initializeCalendar(Locale loc) {
     if (calendar == null) {
         assert loc != null;
         // The format object must be constructed using the symbols for this zone.
         // However, the calendar should use the current default TimeZone.
         // If this is not contained in the locale zone strings, then the zone
         // will be formatted using generic GMT+/-H:MM nomenclature.
         calendar = Calendar.getInstance(loc);
     }
 }
```

这个 Calendar 对象在格式化和解析过程中会被修改

多线程共享同一个 SimpleDateFormat 实例时，会同时操作这个 Calendar 对象

2、数据竞争问题：

竞态条件： 多个线程无控制地并发调用同一个实例的 format/parse 方法，必然导致对 Calendar 状态的读写竞争，结果不可预测。

线程安全解决方案

1、每次创建新实例（开销大）： 在每次需要格式化或解析时，都创建一个新的 SimpleDateFormat 实例。这是最简单的方法，但频繁创建/销毁对象可能带来性能开销，尤其是在高并发场景。

2、使用 ThreadLocal：为每个线程绑定一个独立的 SimpleDateFormat 实例。这是 JDK 8 之前最常用的高效解决方案。

```
public class T1 {
    1 usage
    private static final ThreadLocal<SimpleDateFormat> threadLocalFormat =
            ThreadLocal.withInitial(() -> new SimpleDateFormat( pattern: "yyyy-MM-dd"));
    1 usage
    public String formatDate(Date date) {
        return threadLocalFormat.get().format(date);
    }
    public static void main(String[] args) {
         T1 t=new T1();
        String s = t.formatDate(new Date());
        System.out.println(s);
```

3、同步锁：

使用 synchronized 保证线程安全

```
public class T2 {
    1 usage
    public synchronized String formatDate(Date date) {
        SimpleDateFormat sdf = new SimpleDateFormat( pattern: "yyyy-MM-dd");
        return sdf.format(date);
    }

    public static void main(String[] args) {
        T2 t=new T2();
        String s = t.formatDate(new Date());
        System.out.println(s);
    }
}
```

4、Java 8 新 API：

使用线程安全的 DateTimeFormatter

```java
public class T3 {
    // 定义一个线程安全的、不可变的格式化器
    1 usage
    private static final DateTimeFormatter formatter =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
    public static void main(String[] args) {
        // 格式化 (LocalDateTime 是 java.time 中的日期时间类)
        String s = formatter.format(LocalDateTime.now());
        System.out.println(s);
    }
}
```

面试篇之场景题

49、为什么 ConcurrentHashMap 中

key 不允许为 Null

【腾讯】

在 Java 的 ConcurrentHashMap 中，key 不允许为 null。

1. 避免并发场景下的歧义性

歧义问题：在并发环境中，如果允许 key 为 null，调用 map.get(null) 可能返回两种结果：

Key null 不存在于 Map 中（返回 null）。

Key null 存在，但其对应的 value 是 null（也返回 null）。



并发安全风险：在并发操作中（如 put 和 get 同时发生），无法通过 null 返回值明确区分“key 不存在”和“value 为 null”两种状态，这会导致业务逻辑错误。

对比 HashMap：HashMap 允许 key 为 null，因为它是单线程数据结构，开发者可以自行控制 null 的使用场景。但并发场景下这种歧义会放大风险。

2. 简化并发实现逻辑

原子操作约束：ConcurrentHashMap 的并发安全依赖于 CAS 操作和锁机制。若允许 null key，在实现 get()、put() 等方法时，需要额外处理 null 的特殊情况，这会增加代码复杂度。降低并发性能（需额外检查 null）。引入潜在的错误路径（如未覆盖的边界条件）。

设计哲学：禁止 null key/value 可以使 API 更简洁、可靠，符合并发容器“明确失败”（Fail-Fast）的设计原则。

最佳实践：业务中应避免使用 null 作为 key（即使使用 HashMap），推荐用 Optional

```
Optional<String> optionalKey = Optional.ofNullable(key);
if (optionalKey.isPresent()) {
    // 使用 key 进行操作
} else {
    // 处理 key 为 null 的情况
}
```

面试篇之场景题

50、在 Java 中实现单例模式有哪些方法

【阿里】

1. 饿汉式（Eager Initialization）

线程安全，类加载时立即初始化

```
5 usages
public class EagerSingleton {
    1 usage
    private static final EagerSingleton INSTANCE = new EagerSingleton();

    1 usage
    private EagerSingleton() {} // 私有构造器

    1 usage
    public static EagerSingleton getInstance() {
        return INSTANCE;
    }
}
```

优点：实现简单，线程安全

缺点：可能造成资源浪费（未使用时也加载）

2. 懒汉式（Lazy Initialization）

(1) 基础版（非线程安全）

```
public class LazySingleton {
    3 usages
    private static LazySingleton instance;

    1 usage
    private LazySingleton() {}

    no usages
    public static LazySingleton getInstance() {
        if (instance == null) {
            instance = new LazySingleton(); // 多线程下可能重复创建
        }
        return instance;
    }
}
```

缺点：非线程安全

(2) 同步方法版（线程安全但低效）

```
no usages
public static synchronized LazySingleton getInstance() {
    if (instance == null) {
        instance = new LazySingleton();
    }
    return instance;
}
```

缺点：每次调用都同步，性能差

3. 双重检查锁（Double-Checked Locking）

高效线程安全（需配合 volatile）

```java
public class DCLSingleton {
    4 usages
    private static volatile DCLSingleton instance; // volatile 禁止指令重排序
    1 usage
    private DCLSingleton() {}
    no usages
    public static DCLSingleton getInstance() {
        if (instance == null) { // 第一次检查
            synchronized (DCLSingleton.class) {
                if (instance == null) { // 第二次检查
                    instance = new DCLSingleton();
                }
            }
        }
        return instance;
    }
}
```

优点：线程安全且高效（仅第一次同步）

注意：Java 5+ 的 volatile 可确保可见性和有序性

4. 静态内部类（Static Inner Class）

推荐方案：延迟加载 + 线程安全

```java
public class InnerClassSingleton {
    1 usage
    private InnerClassSingleton() {}

    1 usage
    private static class SingletonHolder {
        1 usage
        static final InnerClassSingleton INSTANCE = new InnerClassSingleton();
    }

    no usages
    public static InnerClassSingleton getInstance() {
        return SingletonHolder.INSTANCE; // 首次调用时加载内部类
    }
}
```

原理：利用类加载机制保证线程安全（JVM 在加载类时互斥）

优点：无锁、高效、懒加载

5. 枚举（Enum）

最佳实践（《Effective Java》推荐）

```
public enum EnumSingleton {
    no usages
    INSTANCE; // 单例实例

    no usages
    public void doSomething() {
        // 业务方法
    }
}
```

调用方式：EnumSingleton.INSTANCE.doSomething()

优点：

天生线程安全

防反射攻击

防序列化破坏

代码简洁

6. ThreadLocal 单例

线程级单例（每个线程唯一）

```java
public class ThreadLocalSingleton {
    1 usage
    private static final ThreadLocal<ThreadLocalSingleton> instance =
        ThreadLocal.withInitial(ThreadLocalSingleton::new);

    1 usage
    private ThreadLocalSingleton() {}

    no usages
    public static ThreadLocalSingleton getInstance() {
        return instance.get();
    }
}
```

适用场景：需要线程隔离的单例（如数据库连接）

枚举是最健壮的实现，能抵御反射和序列化破坏，优先推荐使用。